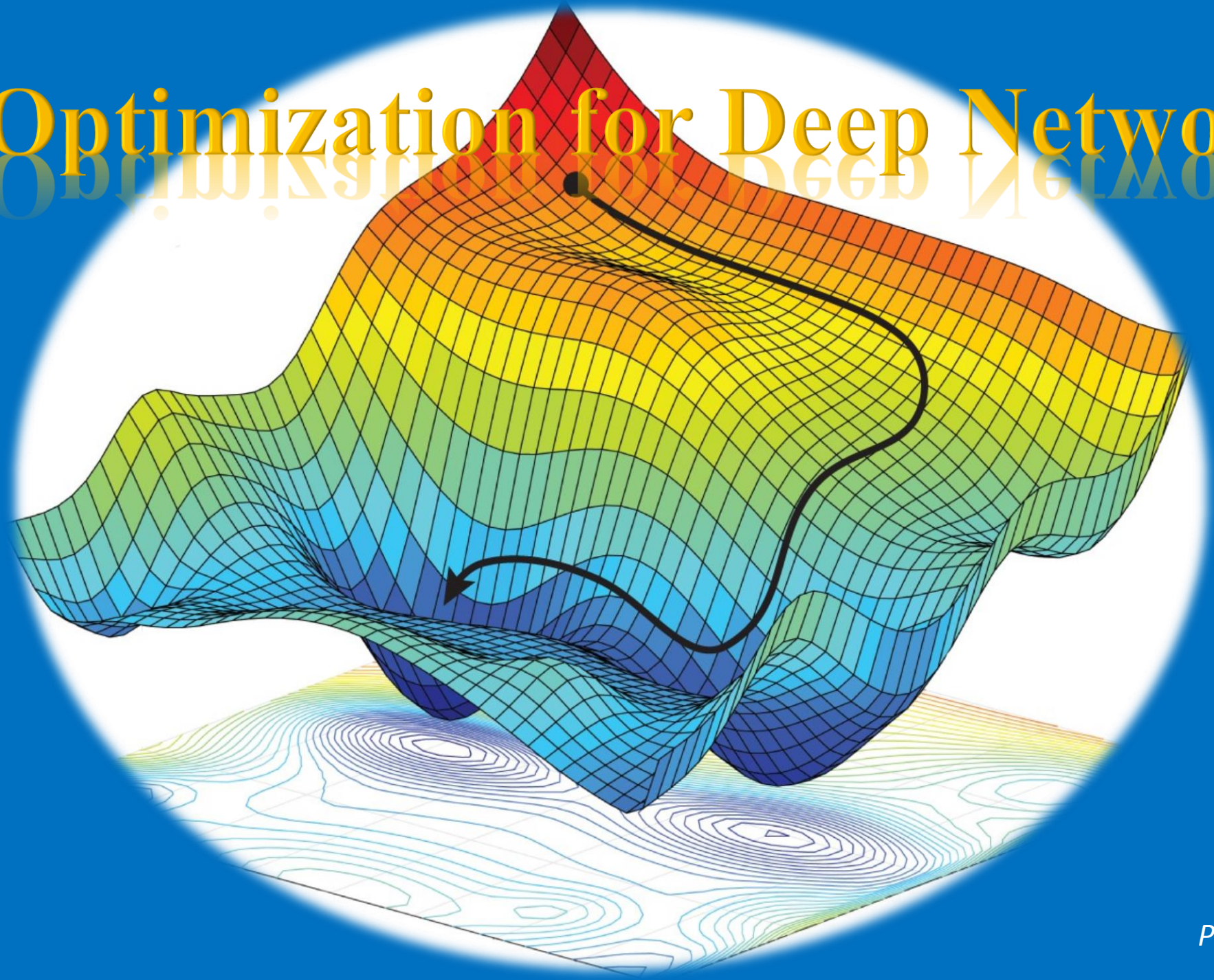
Optimization for Deep Networks



Pr. BEN LAHMAR EL HABIB

Agenda

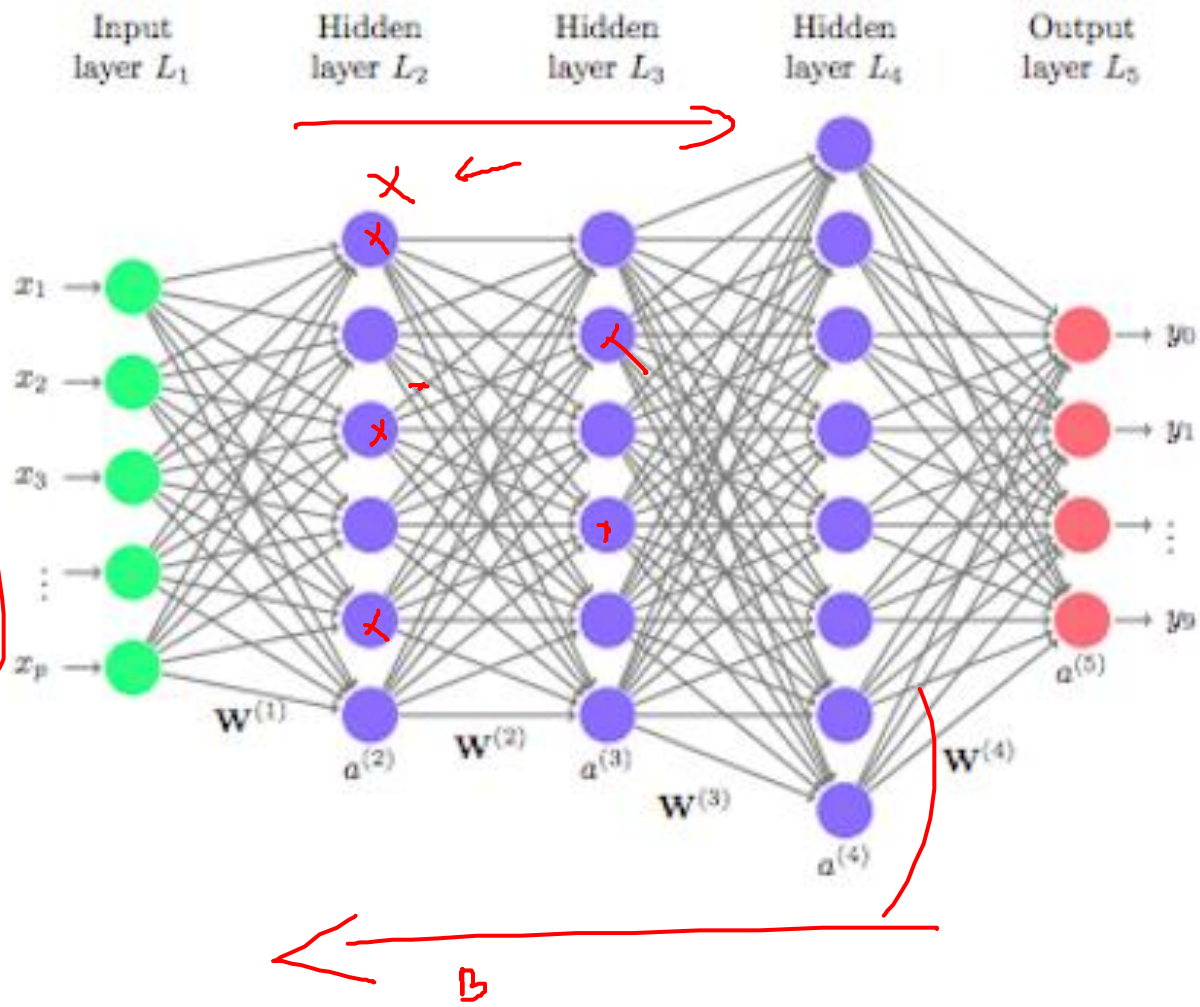
- Vanilla SGD
- Mini batch
- Momentum
- Adaptive Learning Rates
  - NAG
  - Rprop
  - AdaGrad
  - RMSProp
  - AdaDelta
  - Adam
  - AdaMax
  - Nadam
  - AMSGrad
  - Autres

Rappel

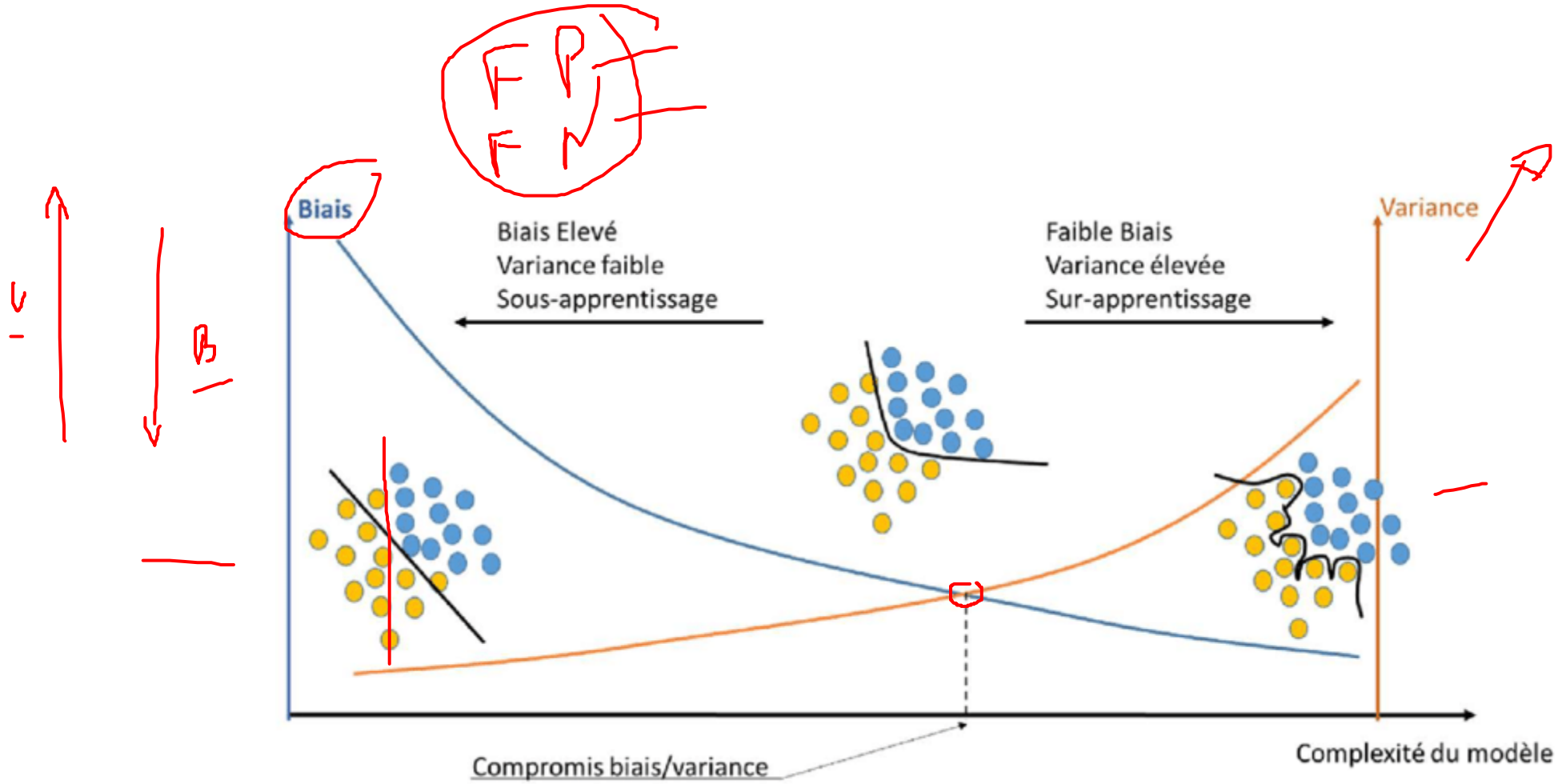
Three questions:

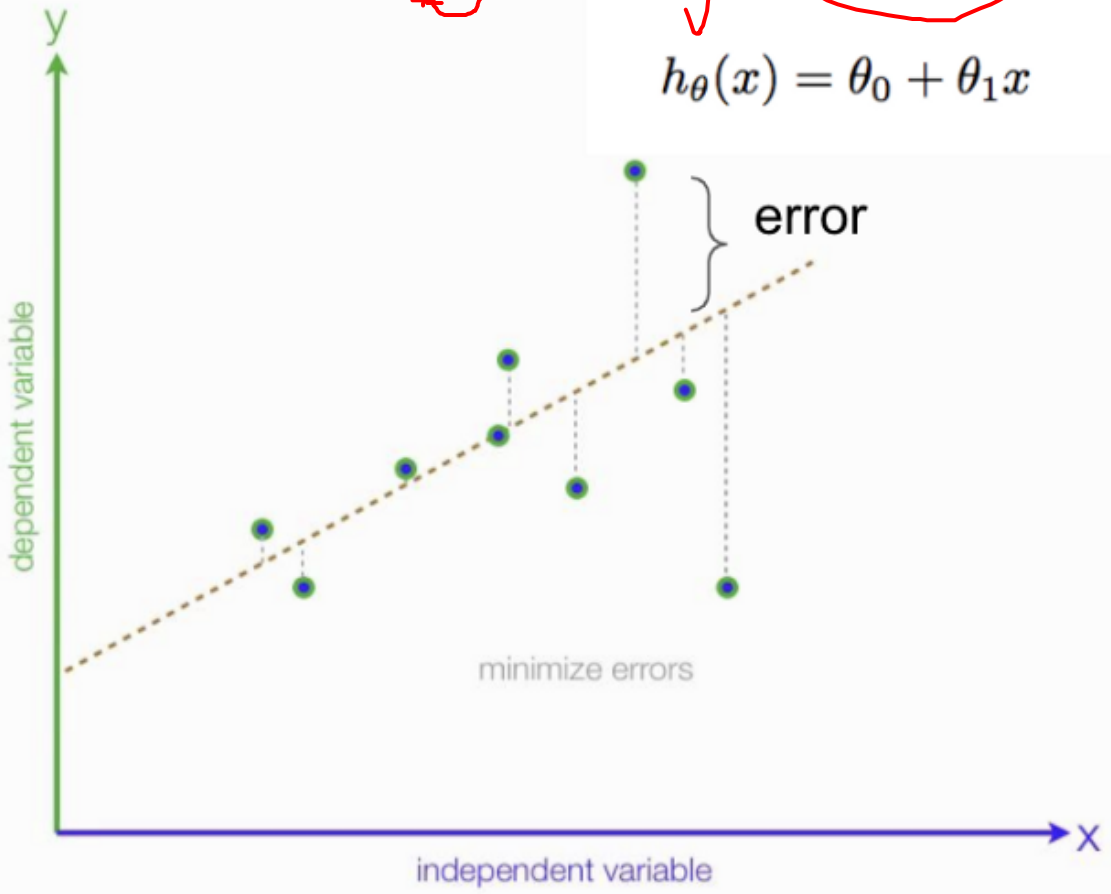
1. What architecture $F(W,X)$.
2. What loss function $L(W, y^i, X^i)$.
3. What optimization method.

- DG: $w_{t+1} = w_t - \alpha \frac{\partial L}{\partial w_t}$
- Bais & Variance
- Overfetting & underfetting
- Regularization
- Explosed - vanished



Variance and biais





$$h_\theta(x) = \theta_0 + \theta_1 x$$

Hypothesis:

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Parameters:

$$\theta_0, \theta_1$$

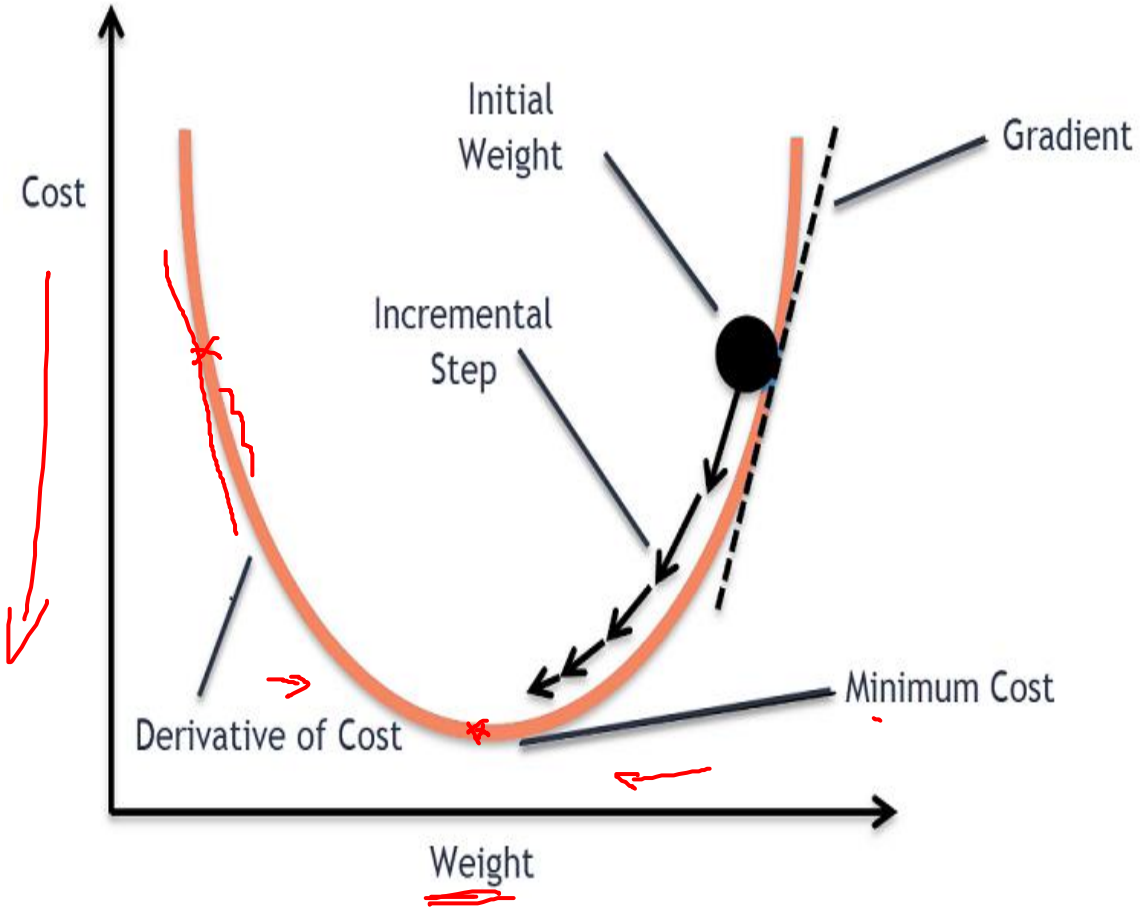
Cost Function:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

Goal:

$$\underset{\theta_0, \theta_1}{minimize} J(\theta_0, \theta_1)$$

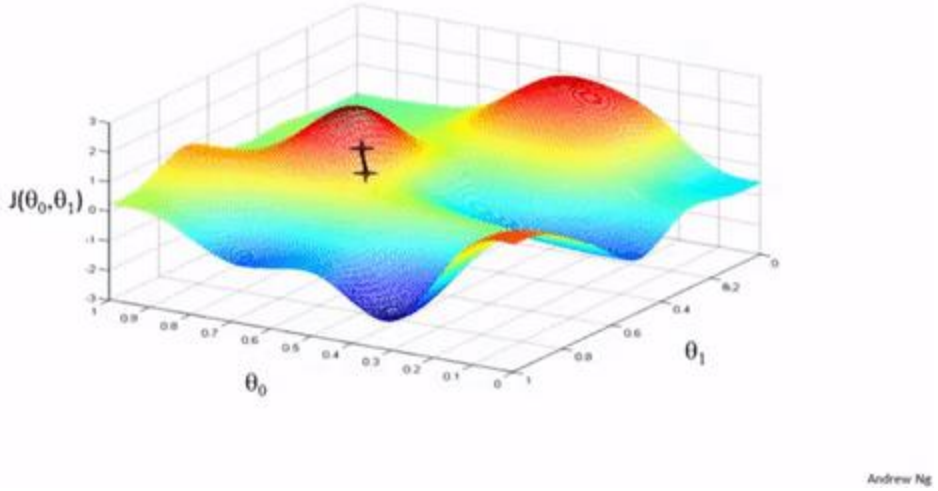
Gradient (Steepest) Descent

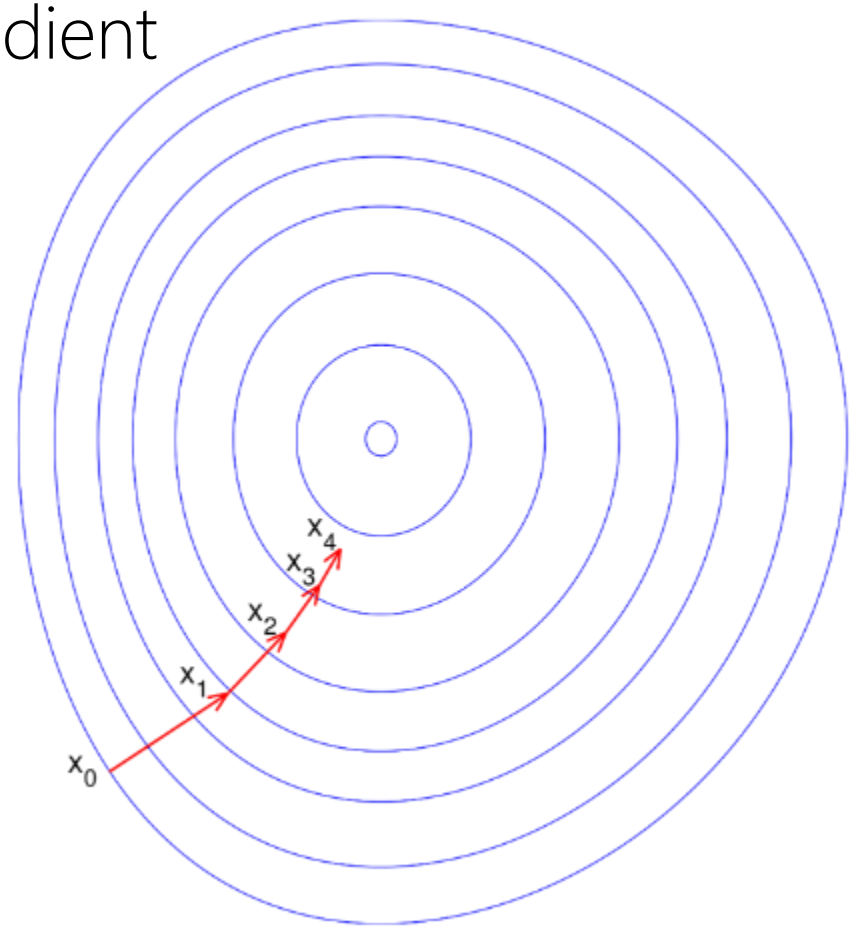


- Calculer la pente (gradient) qui est la dérivée du premier ordre de la fonction au point courant
- Déplacement dans la direction opposée de l'augmentation de la pente à partir du point actuel de la valeur calculée

Gradient (Steepest) Descent

- Se déplacer dans la direction opposée du gradient





Voir : https://remykarem.github.io/backpropagation-demo/

Batch Gradient Descent

- Dans Batch Gradient Descent, toutes les données d'entraînement sont prises en compte pour effectuer une seule étape.
- Nous prenons la moyenne des gradients de tous les exemples d'entraînement, puis utilisons ce gradient moyen pour mettre à jour nos paramètres. Ce n'est donc qu'une étape de descente de gradient en une seule epoch.

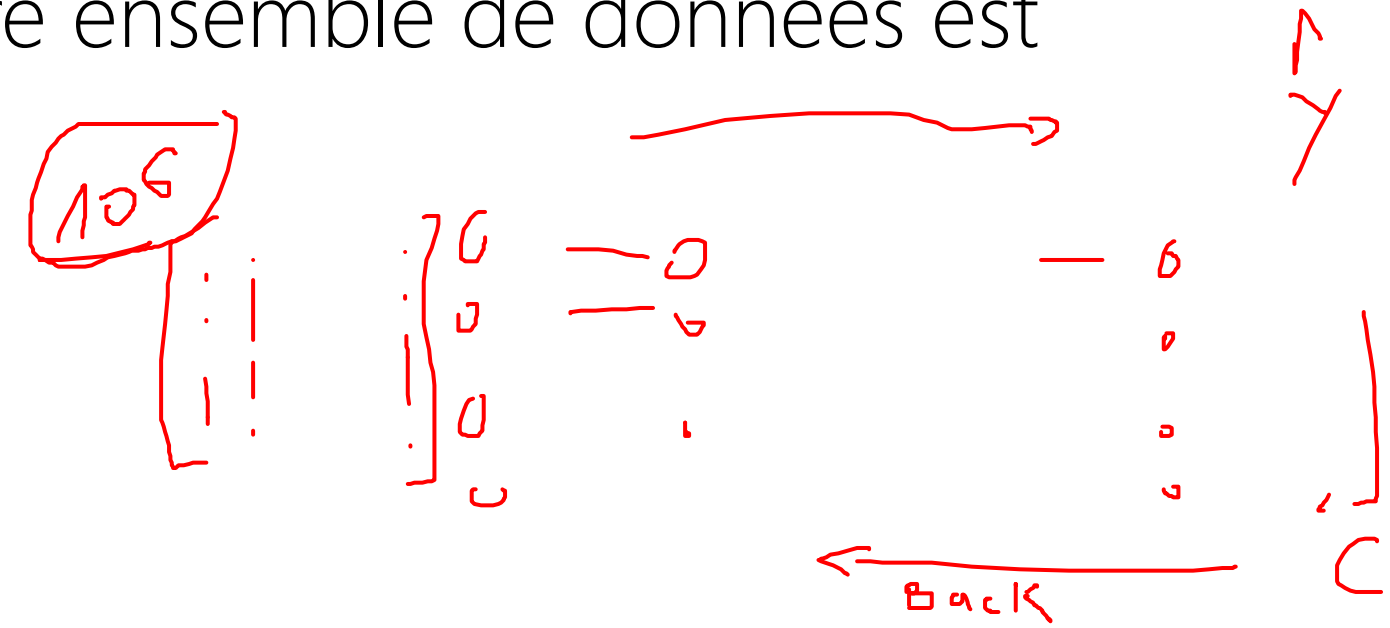


- La descente de gradient par lots est idéale pour les variétés d'erreur convexes ou relativement lisses.



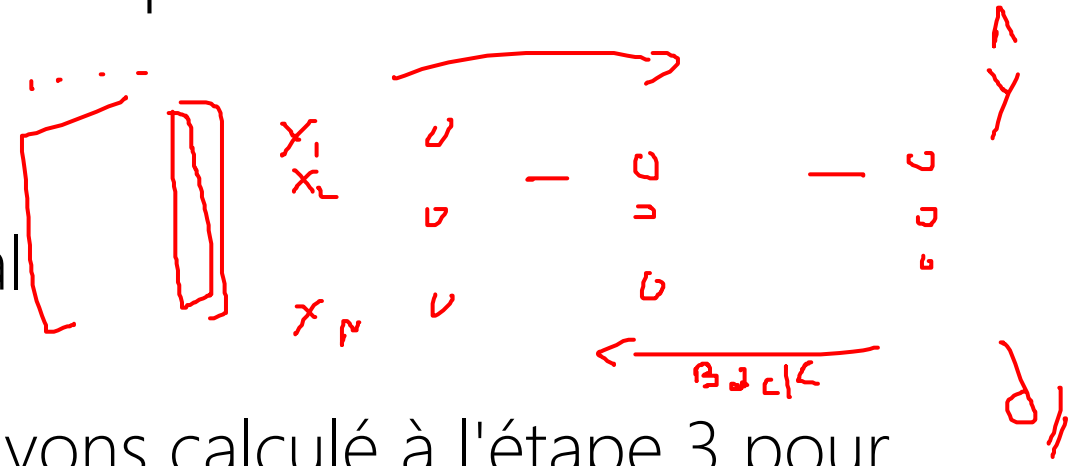
Stochastic Gradient Descent (SGD)

- Dans Batch Gradient Descent, nous avons considéré tous les exemples pour chaque étape de Gradient Descent.
- Mais que faire si notre ensemble de données est très énorme.



Stochastic Gradient Descent (SGD)

- Dans Stochastic Gradient Descent (SGD), nous ne considérons qu'un seul exemple à la fois pour faire un seul pas. Nous effectuons les étapes suivantes en une seule époque pour SGD:



1. Prenons un exemple
2. Envoyez-le au réseau neuronal
3. Calculer son gradient
4. Utilisez le gradient que nous avons calculé à l'étape 3 pour mettre à jour les poids
5. Répétez les étapes 1 à 4 pour tous les exemples de l'ensemble de données d'entraînement

Properties of Loss function for SGD

La fonction de perte sur tous les échantillons doit se décomposer en une fonction de perte par échantillon

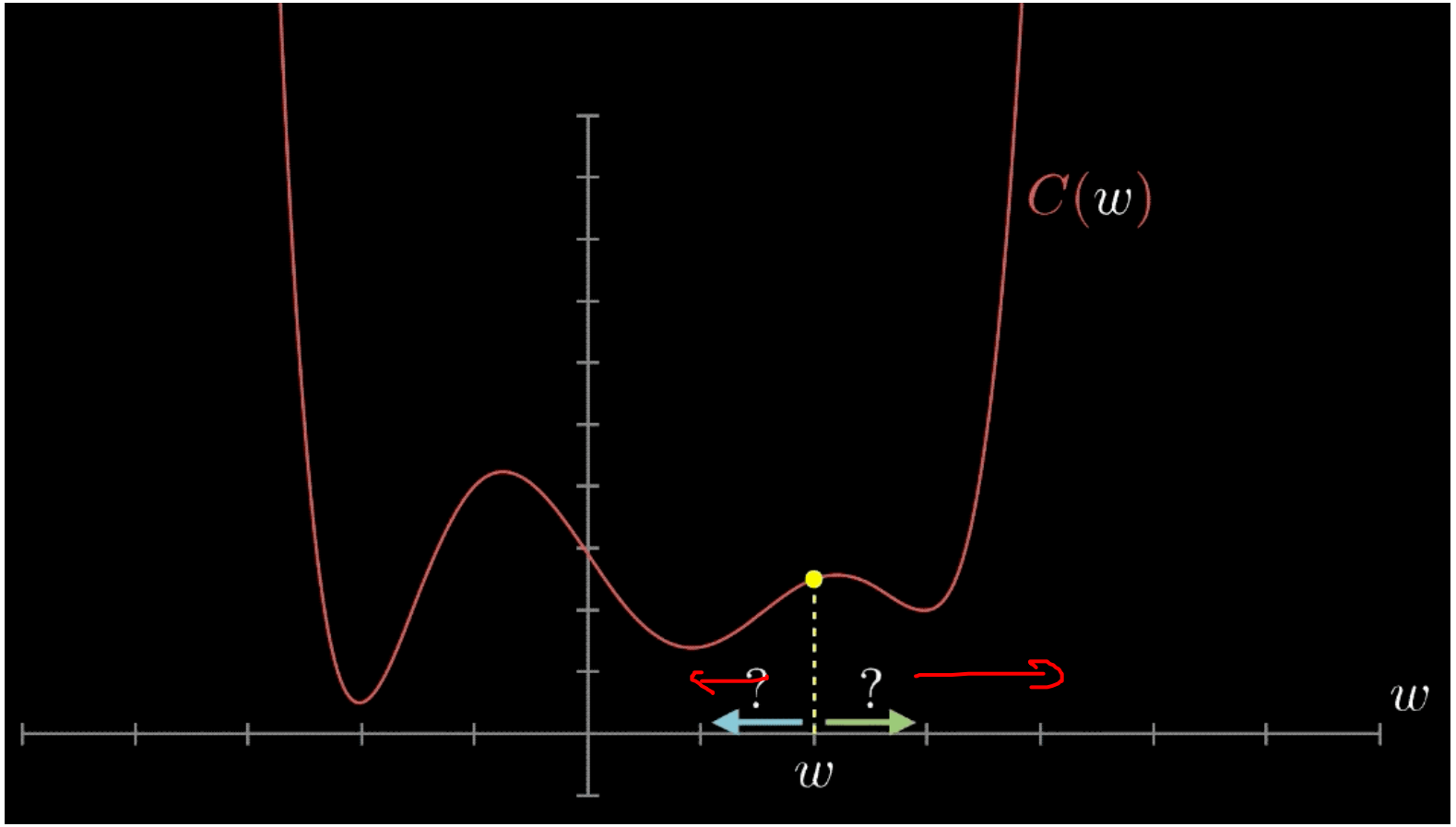
$$f = \sum_{i=0}^{n} (x_i - y_i)^2$$

$$MSE = \frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{n}$$



Vanilla SGD

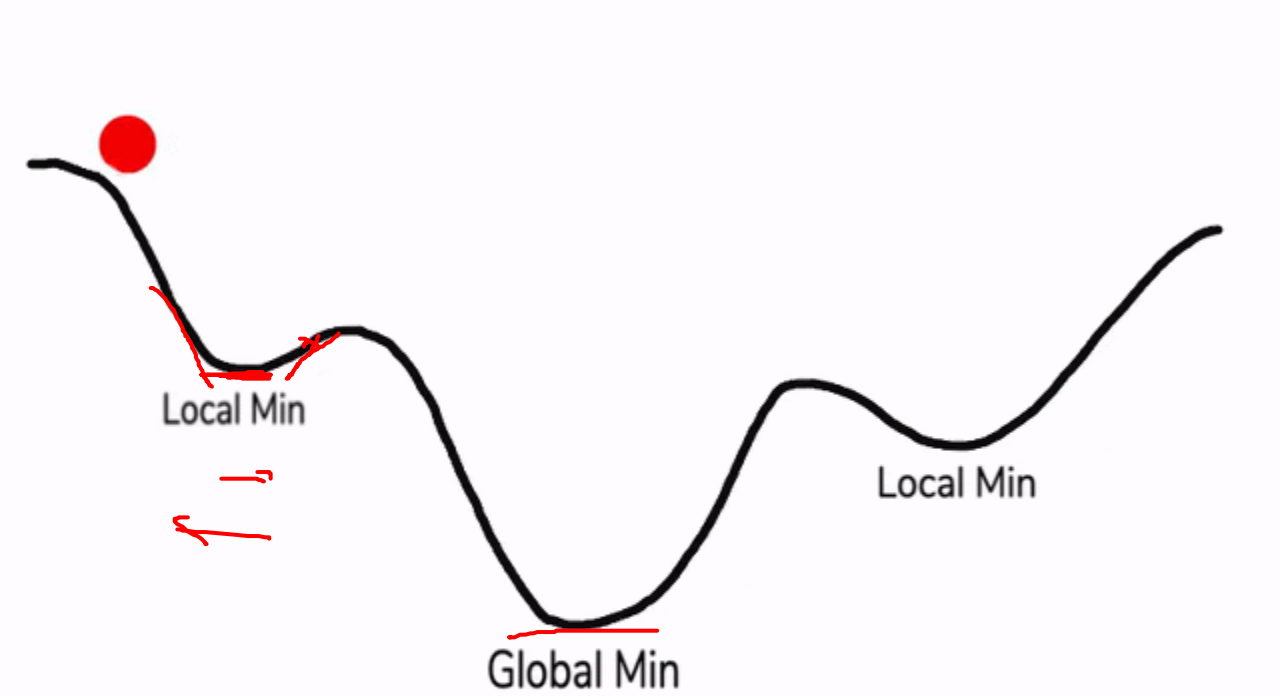
```
for i in range(n_epochs):
    shuffled_data = np.random.shuffle(data)
    for x,y in shuffled_data:
            # Using backpropagation to calculate gradients (change)
            change = compute_gradient(cost_func, x, y, param)
            param = param - l_rate * change
```



$$w_{t+1} = w_t - \alpha \frac{\partial L}{\partial w_t}$$

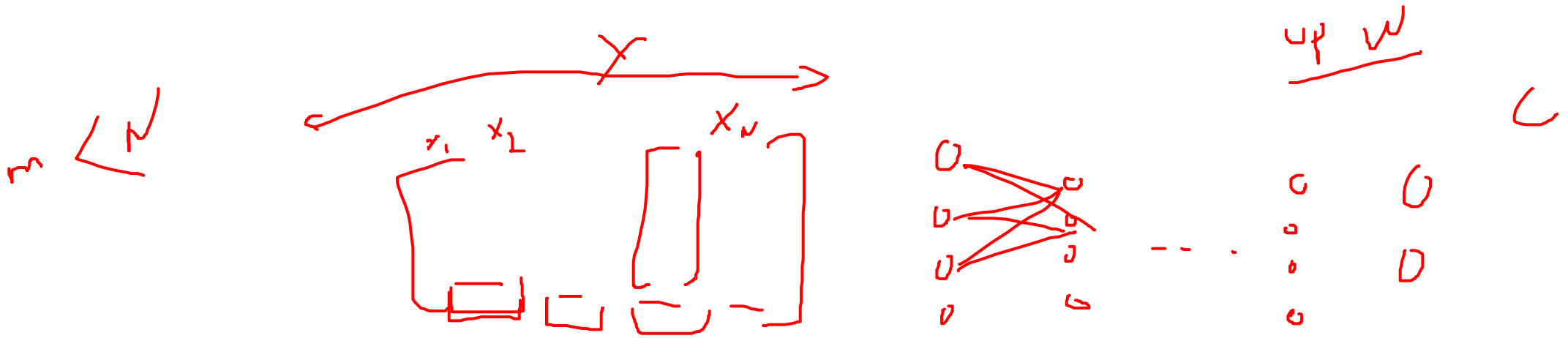
w	Parameters of Network
L	Function of network parameters
t	Iteration number
α	Step size/Learning rate

Probleme

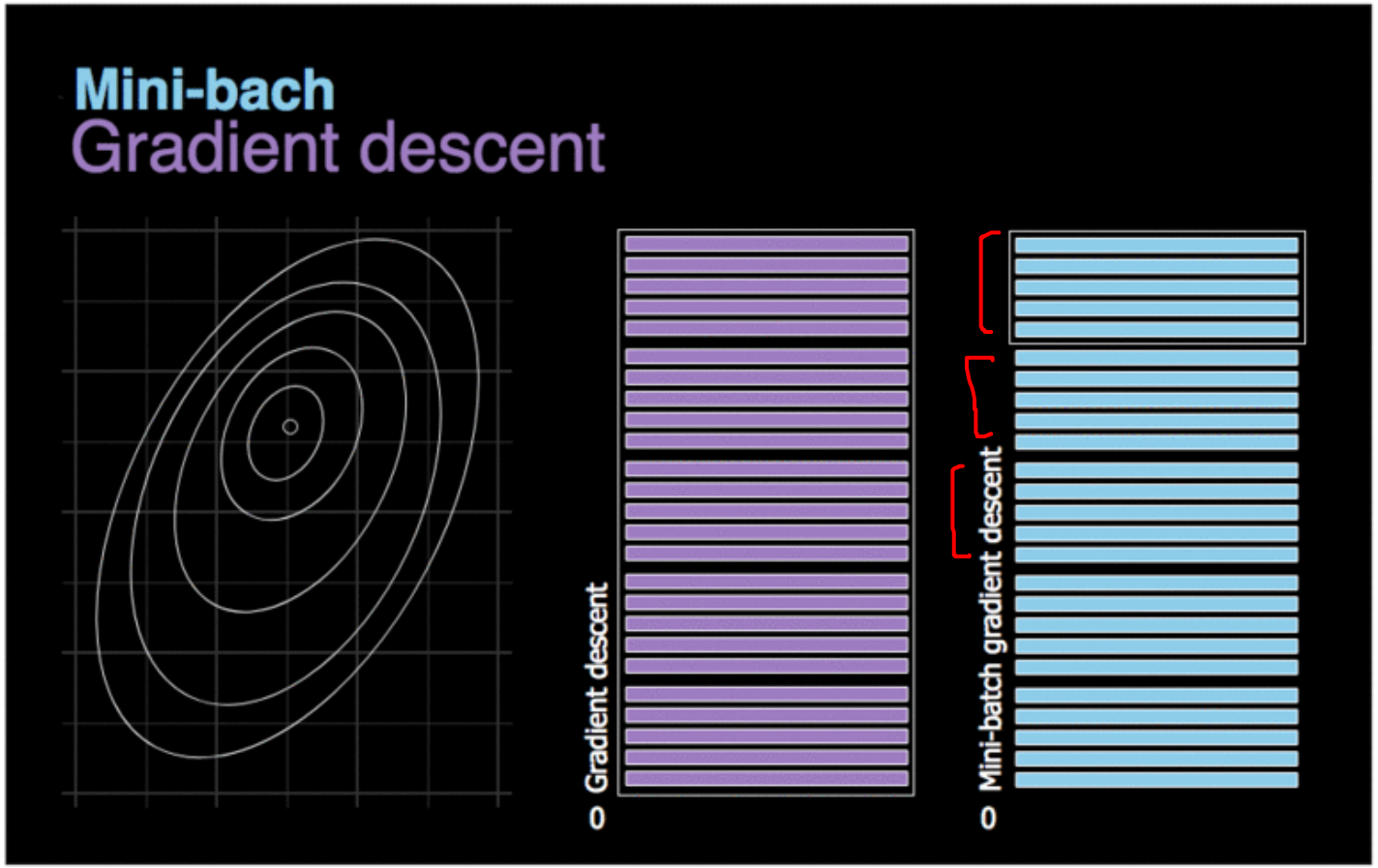


Mini-Batch

Nous n'utilisons pas tous les ensembles de données à la fois ni nous n'utilisons l'exemple unique à la fois. Nous utilisons un lot d'un nombre fixe d'exemples d'entraînement qui est inférieur au jeu de données réel et l'appelons un mini-lot. Cela nous aide à profiter des avantages des deux anciennes variantes que nous avons vues.

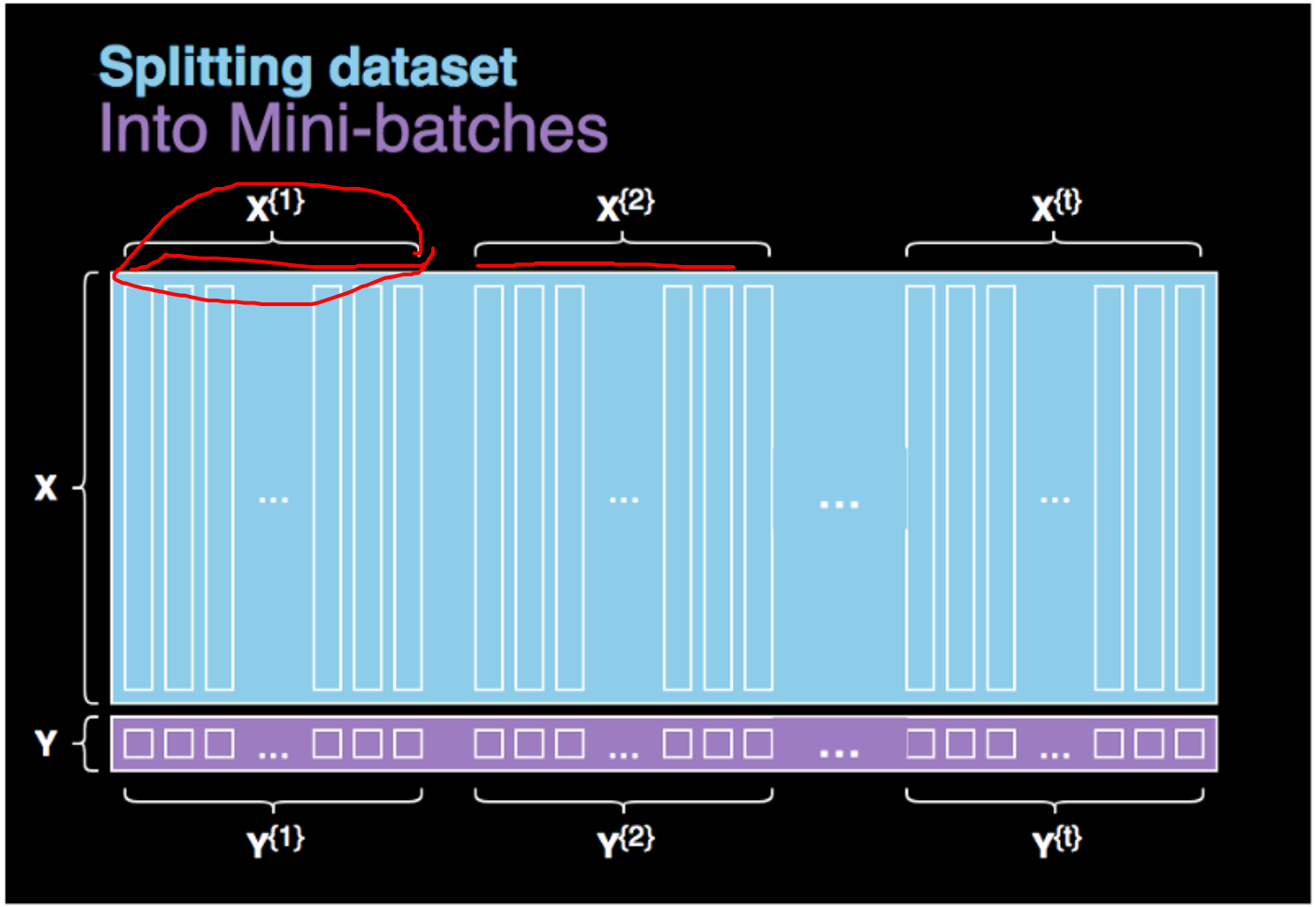


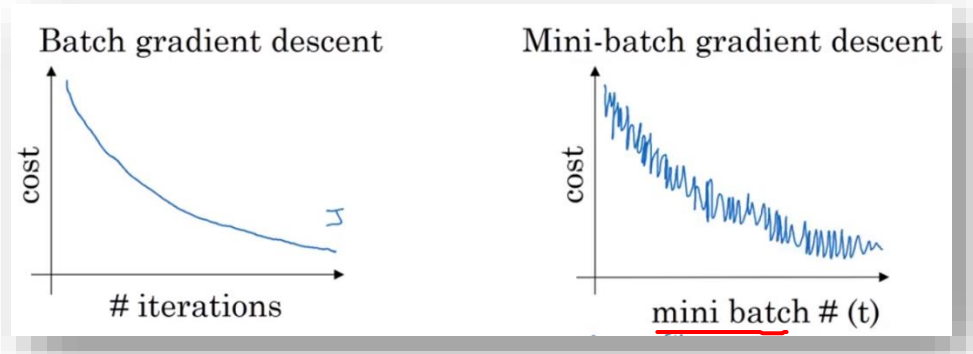
Mini-Batch



Mini-batch gradient descent is a variation of the gradient descent algorithm that splits the training dataset into small batches that are used to calculate model error and update model coefficients.

Mini-Batches splitting





https://towardsdatascience.com/how-to-train-neural-network-faster-with-optimizers-d297730b3713

EQUATION

$$W = W - \alpha \nabla J(W, b, x^{(z:z+bs)}, y^{(z:z+bs)})$$

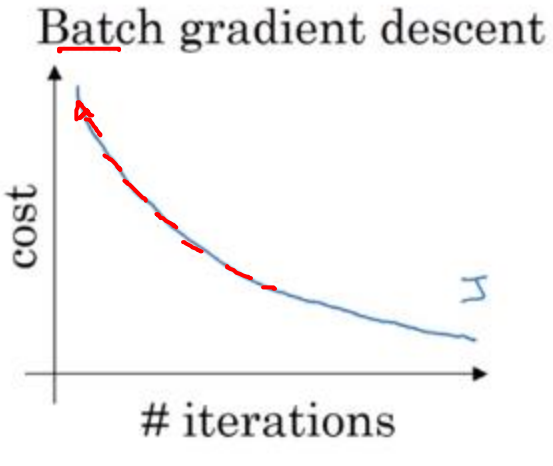
Where bs is the mini-batch size and the cost function is:

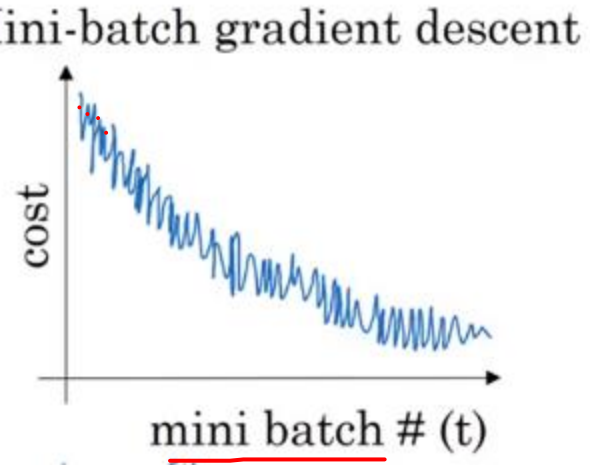
$$J(W, b, x^{(z:z+bs)}, y^{(z:z+bs)}) = \frac{1}{bs} \sum_{z=0}^{bs} J(W, b, x^{(z)}, y^{(z)})$$

Mini-Batch

Quel est l'avantage de le faire de cette façon?

1. Premièrement, il atténue une partie du bruit dans SGD, mais pas la totalité, ce qui permet toujours de «sortir» des minimums locaux de la fonction de coût.
2. Deuxièmement, la taille du mini-lot est encore petite, conservant ainsi les avantages de performance de SGD.



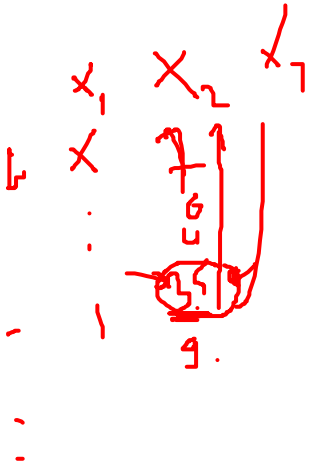




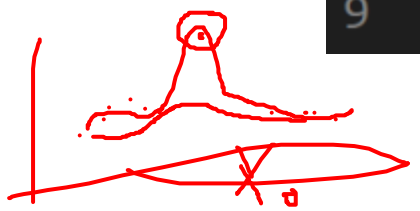
TO READ

The Impact of the Mini-batch Size on the Variance of Gradients in Stochastic Gradient Descent
https://arxiv.org/abs/2004.13146

CODE EXAMPLE

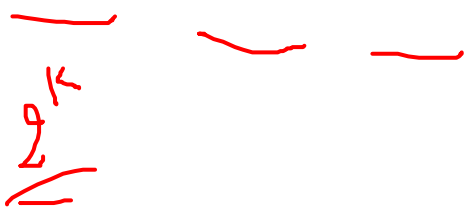


```python
## BEN LAHMAR EL HABIB
from numpy import random
def get_mini_batches(X, y, batch_size):
    random_idxs = random.choice(len(y), len(y), replace=False
    X_shuffled = X[random_idxs,:]
    y_shuffled = y[random_idxs]
    mini_batches = [(X_shuffled[i:i+batch_size,:], y_shuffled
                     i in range(0, len(y), batch_size)]
    return mini_batches
```





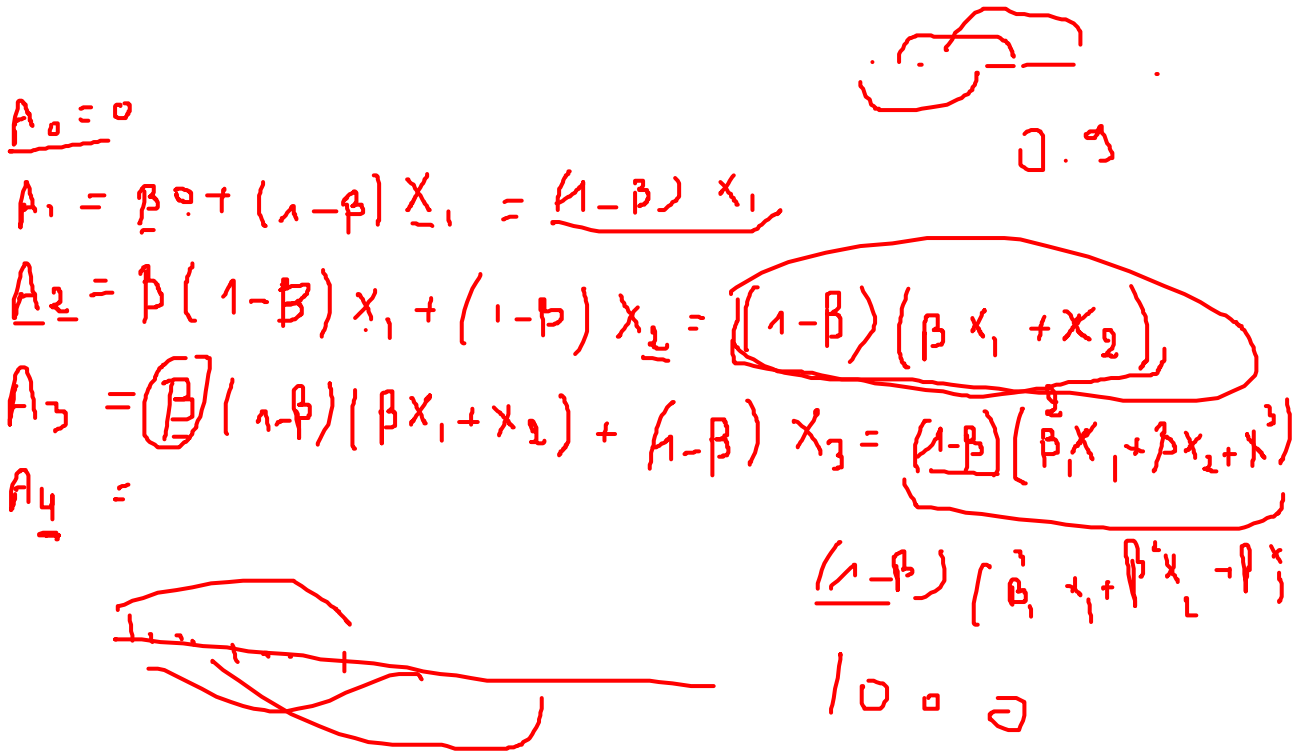




Exponantial Moving Average

Simple moving average = (P1 + P2 + P3 + P4 + ... + Pn) / n

Calcul de la moyenne mobile pondérée = (P_{t-1} * facteur de pondération) + P_t* (facteur de pondération-1)



La moyenne mobile est la moyenne calculée sur ***n*** valeurs successives plutôt que sur l'ensemble des valeurs. Mathématiquement, il est noté:

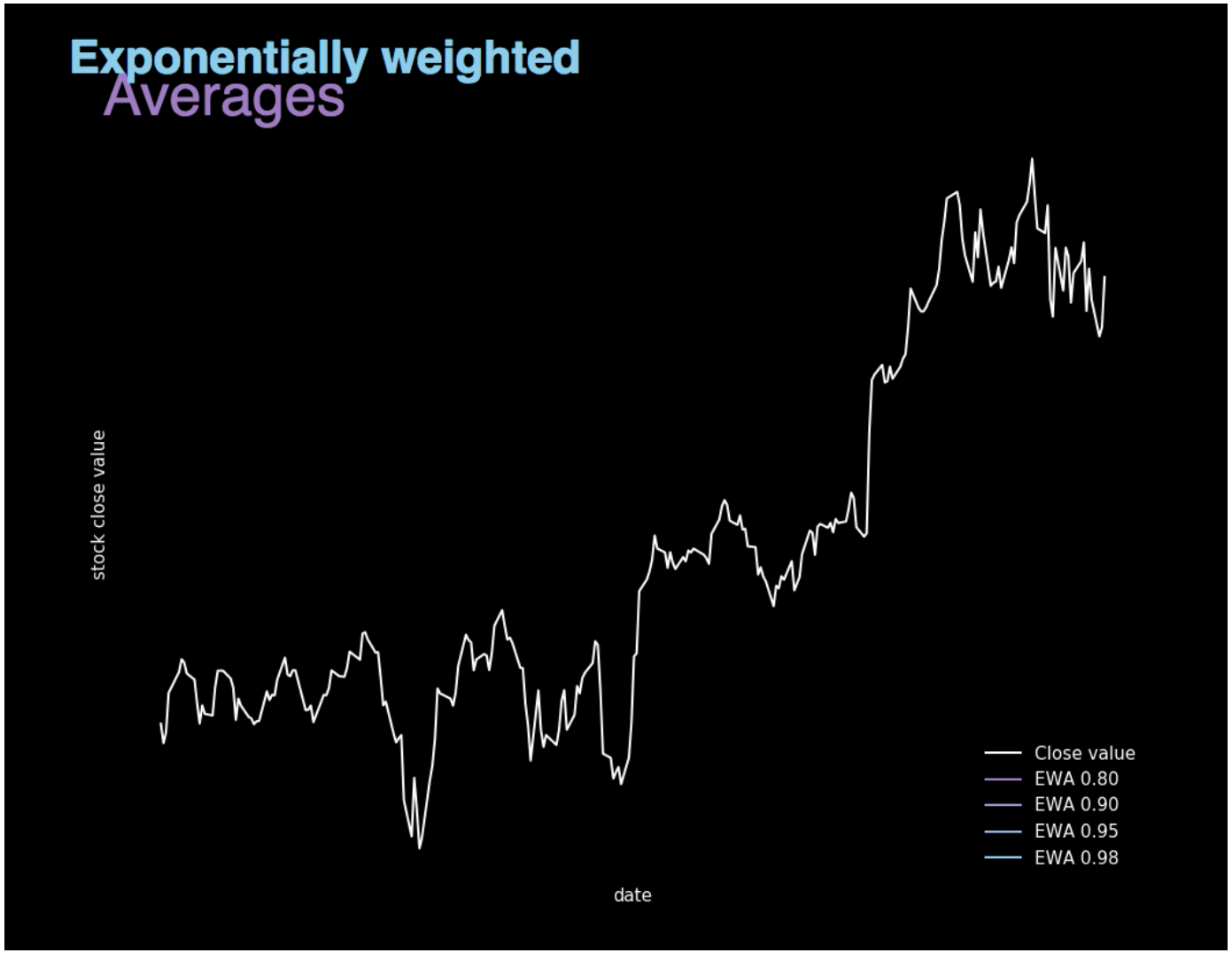
$$A_t = \beta A_{t-1} + (1-\beta) X_t$$

A [i] représente la moyenne mobile au ***i*** point de données pour la valeur ***X [i]*** . Le paramètre ***β*** contrôle la valeur ***n*** sur laquelle la moyenne est calculée.

Par exemple, si β = 0,9, la moyenne mobile considère 10 valeurs successives pour calculer la moyenne; si β = 0,99, la moyenne mobile considère 100 valeurs consécutives pour calculer la moyenne. En général, la valeur n peut être approximée par la formule suivante:

$$n = \frac{1}{1-\beta}$$

Exponantial Moving Average



SGD + Momentum

Accelerating Gradient Descent (Use Momentum)

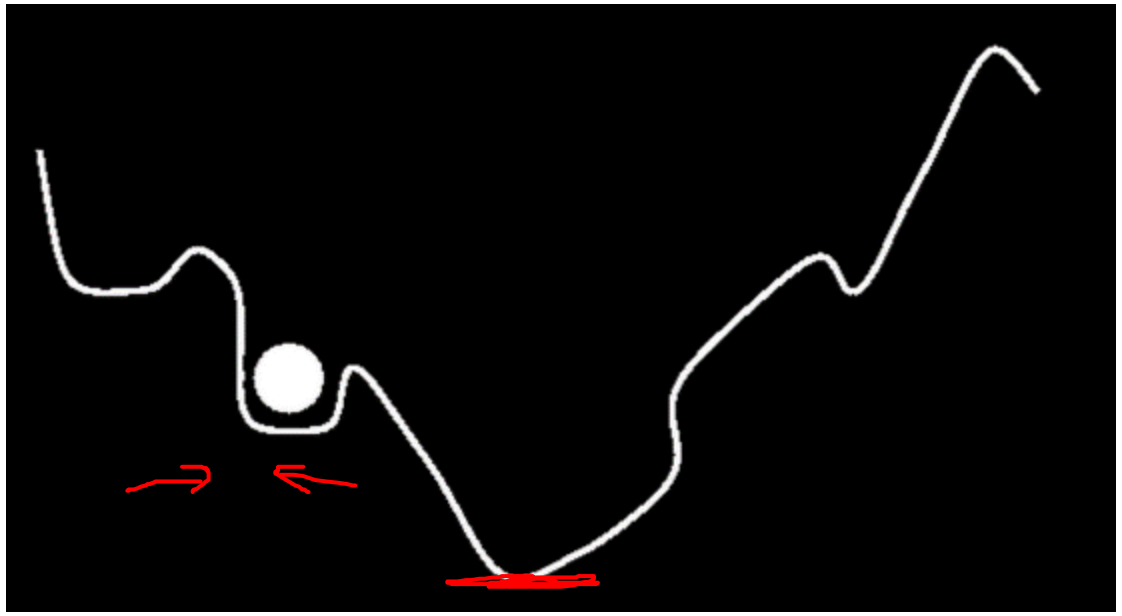
Prenons une situation où vous vous rendez dans un centre commercial récemment ouvert dans une zone inconnue.

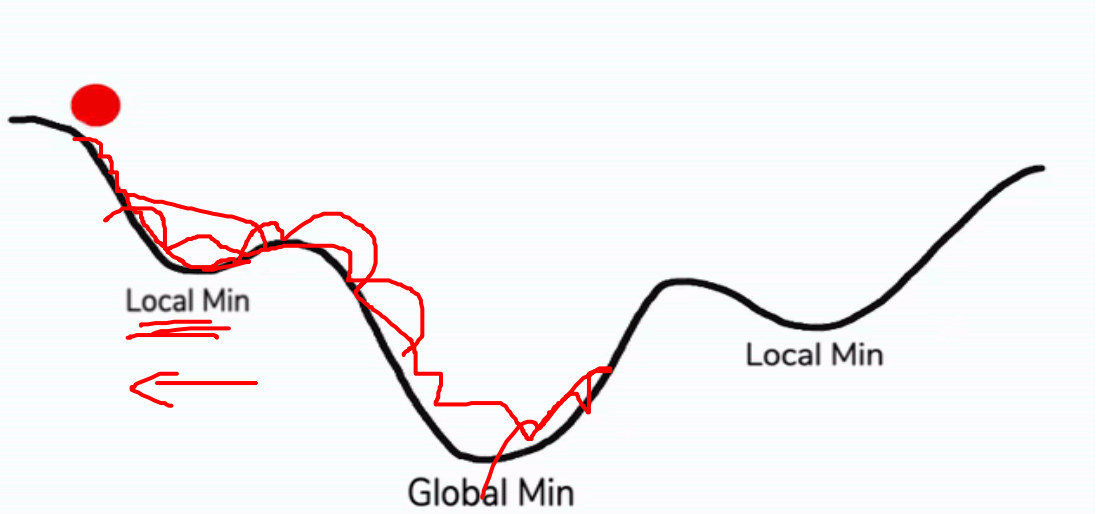
Pendant que vous essayiez de localiser le centre commercial, vous avez demandé à plusieurs personnes l'emplacement du centre commercial et tout le monde vous a demandé de vous rendre au même endroit.

Parce que tout le monde vous dirige vers la même direction, vous irez de plus en plus vite dans cette direction avec plus de confiance. Nous allons maintenant utiliser la même intuition dans la descente de gradient basée sur l'élan(**momentum)**

Motivation for momentum

chaque fois que nous faisons rouler la balle sur la colline (pour chaque époque), elle roule plus vite vers les minima locaux dans l'itération suivante. Cela nous rend plus susceptibles d'atteindre de meilleurs minima locaux (ou peut-être un minima global).



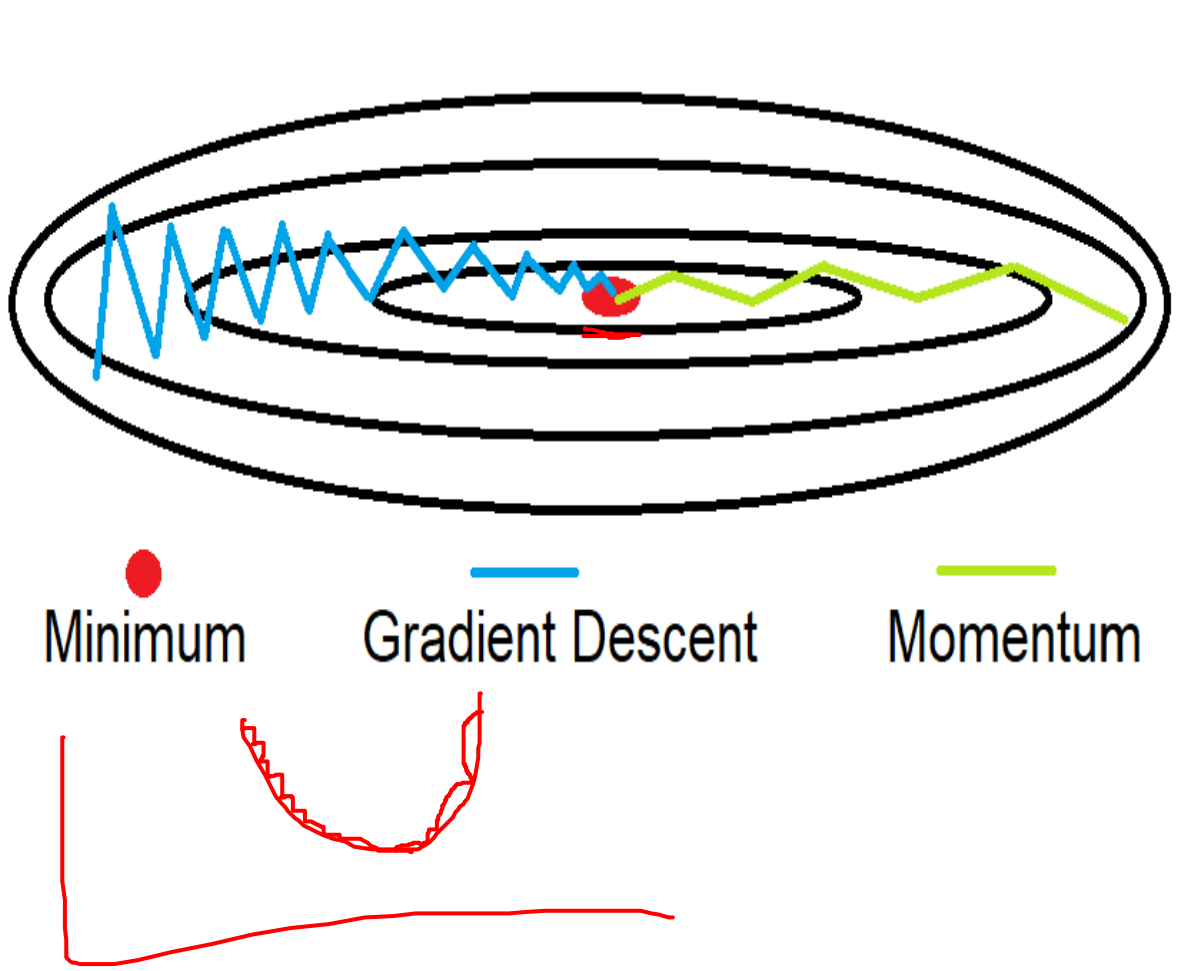


SGD + Momentum

Accelerating Gradient Descent (Use Momentum)

Le chemin suivi par Gradient Descent prend trop de pas par rapport à Momentum. En effet, la descente de gradient oscille trop sur l'axe y et se déplace très moins sur l'axe x, c'est-à-dire vers le minimum.

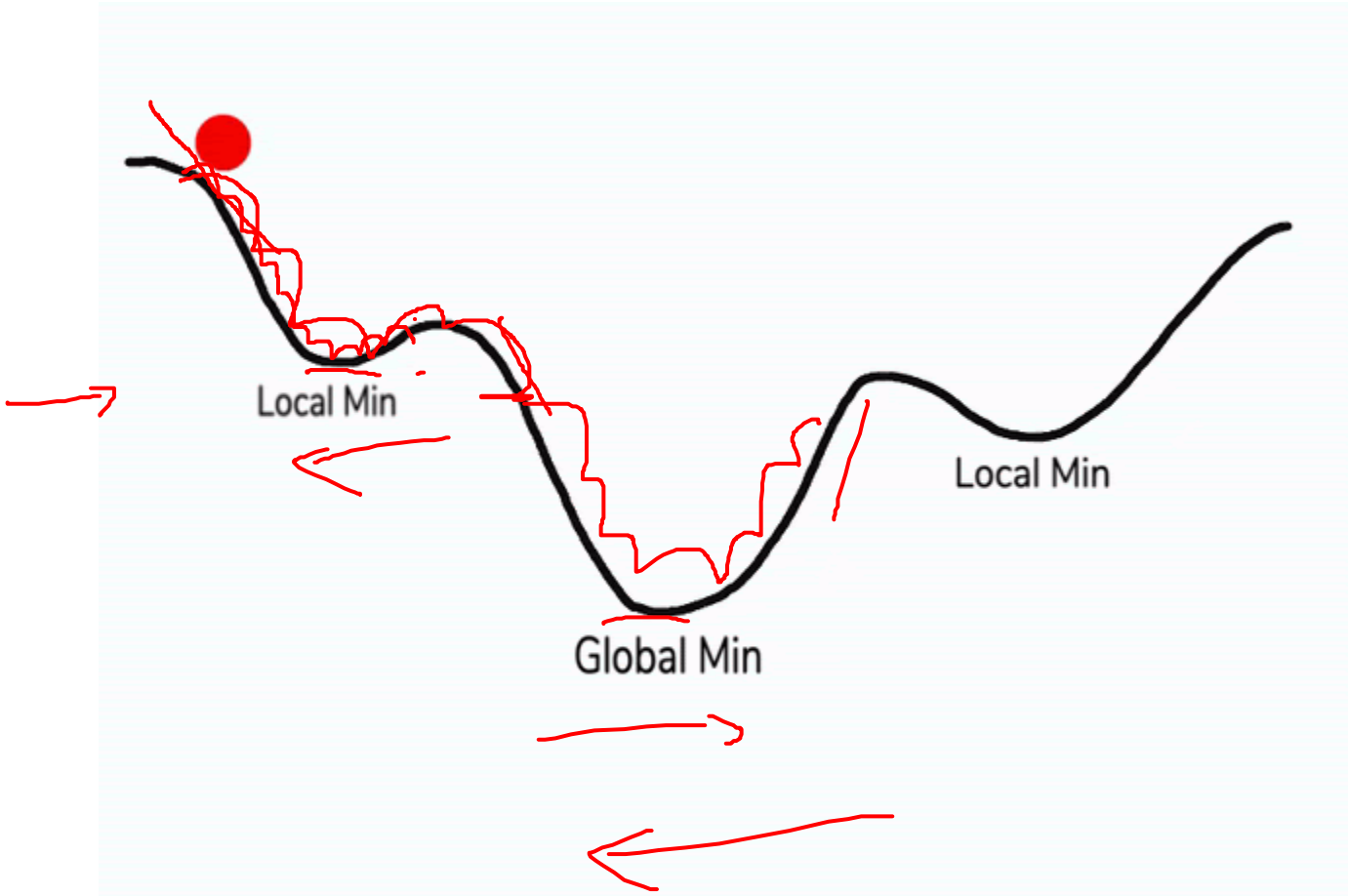
Une bonne solution serait de réduire les oscillations en amortissant le mouvement sur l'axe y. C'est là que la moyenne mobile entre en jeu.



SGD + Momentum

- L'idée de base de Gradient Descent avec Momentum est de calculer la moyenne pondérée exponentiellement des gradients, puis d'utiliser ce gradient pour mettre à jour les poids.

- Il fonctionne plus rapidement que l'algorithme normal pour la descente de gradient.


Local Min
Local Min
Global Min

SGD + Momentum



SGD: $$w_{t+1} = w_t - \alpha \frac{\partial L}{\partial w_t}$$

$$w_{t+1} = w_t - \alpha m_t$$

$$m_t = \beta m_{t-1} + (1-\beta)\frac{\partial L}{\partial w_t}$$

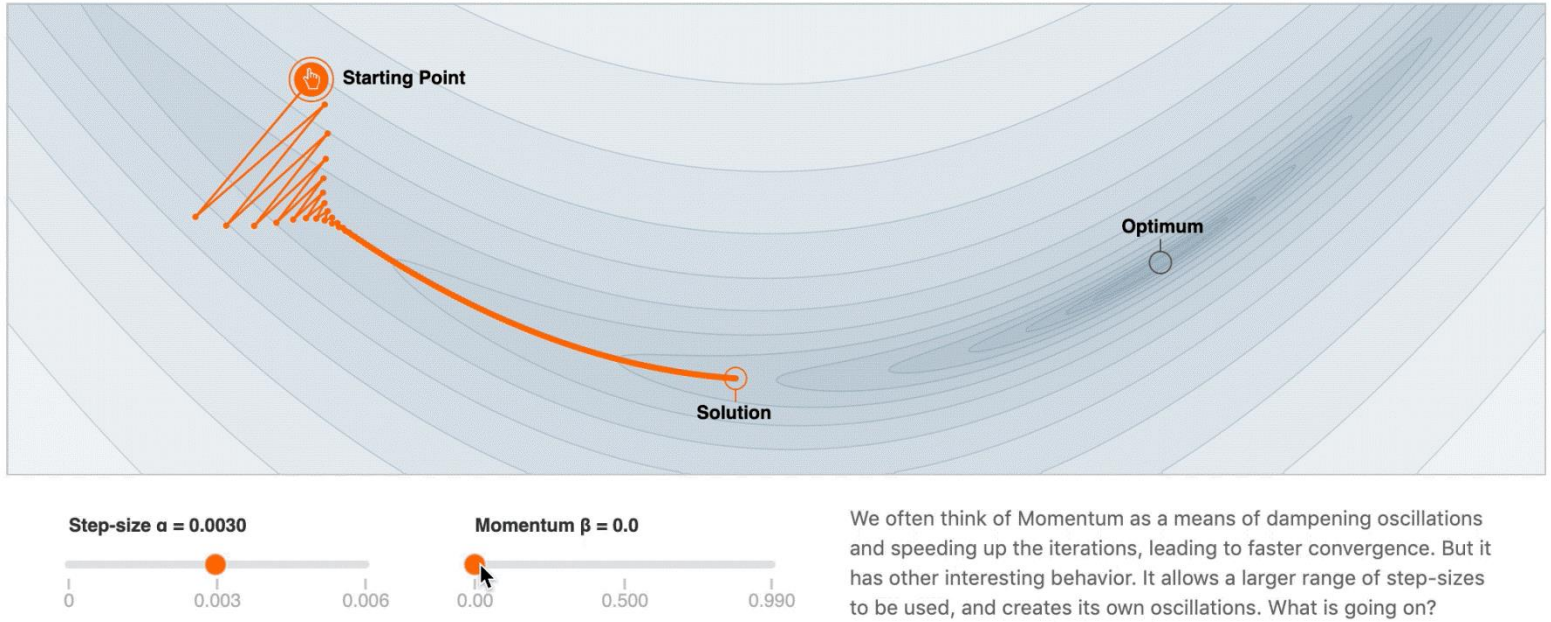
w	Parameters of Network
t	Iteration number
α	Step size/Learning rate
β	Momentum

Comment choisir Beta?

- Le momentum (bêta) doit être plus élevé pour lisser la mise à jour car nous accordons **plus de poids** aux gradients passés.
- L'utilisation de la valeur par défaut β = 0,9 est suggérée mais peut être réglée entre 0,8 et 0,999 si nécessaire.
- Momentum prend en compte les gradients passés afin de lisser les mesures de la gradient. Il peut être mis en œuvre avec descente gradient batch, descente gradient mini-batch ou descente gradient stochastique.

SGD + Momentum

- At iteration t you add updates from previous iteration t_{-n} by weight β^n
- You effectively multiply your updates by $\frac{1}{1-\beta}$



https://distill.pub/2017/momentum/

Exemple de code

```python
def momentum(model, X_train, y_train, minibatch_size):
    velocity = {k: np.zeros_like(v) for k, v in model.items()}
    gamma = .9
    #print(velocity)
    minibatches = get_minibatch(X_train, y_train, minibatch_size)

    for iter in range(1, n_iter + 1):
        idx = np.random.randint(0, len(minibatches))
        X_mini, y_mini = minibatches[idx]

        grad = get_minibatch_grad(model, X_mini, y_mini)

        for layer in grad:
            velocity[layer] = gamma * velocity[layer] + alpha * grad[layer]
            model[layer] -= velocity[layer]

    return model
```

$$V_t = \beta V_{t-1} + \alpha \nabla_w L(W, X, y)$$
$$W = W - V_t$$

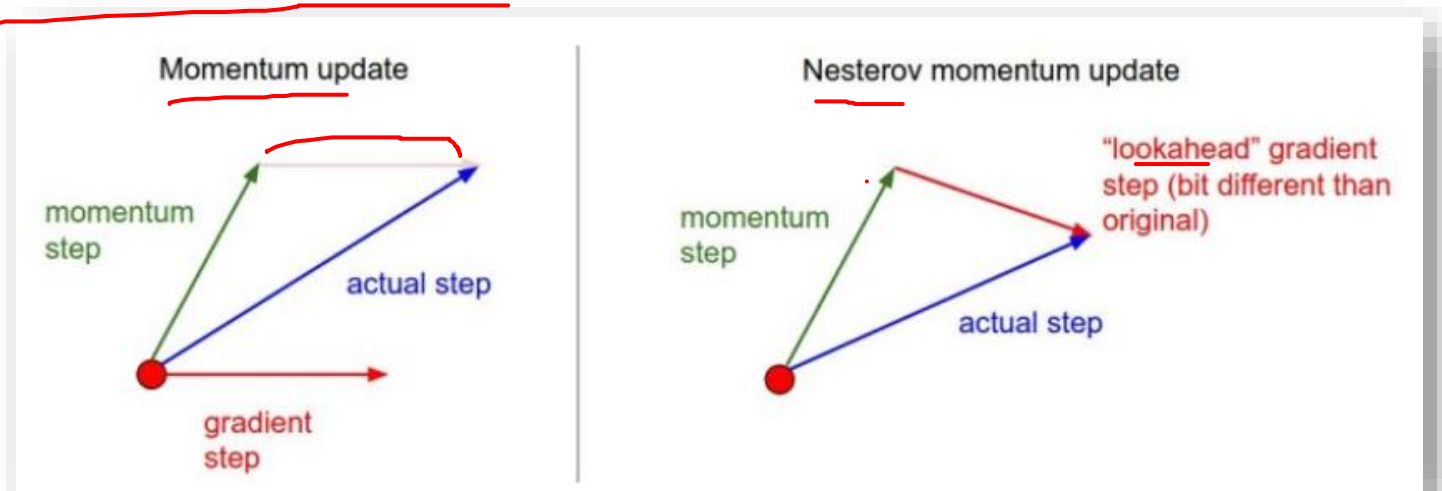
```python
# Momentum update for each parameter
    for l in range(L):
        # compute velocities
        v["dW" + str(l+1)] = beta*v["dW" + str(l+1)] + (1-beta)*grads['dW' + str(l+1)]
        v["db" + str(l+1)] = beta*v["db" + str(l+1)] + (1-beta)*grads['db' + str(l+1)]
        # update parameters
        parameters["W" + str(l+1)] = parameters["W" + str(l+1)] -learning_rate*v["dW" + str(l+1)]
        parameters["b" + str(l+1)] = parameters["b" + str(l+1)] -learning_rate*v["db" + str(l+1)]
```

$$w_{t+1} = w_t - \alpha m_t$$

$$m_t = \beta m_{t-1} + (1-\beta)\frac{\partial L}{\partial w_t}$$

Nesterov Accelerate Gradient (NAG)

- L'intuition derrière NAG peut être mise en une seule phrase:
  - Regardez devant vous avant de sauter!



- Ilya Sutskever, 2012

- Faites d'abord un saut comme momentum
- Ensuite, selon l'endroit où vous arrivez, corrigez les paramètres

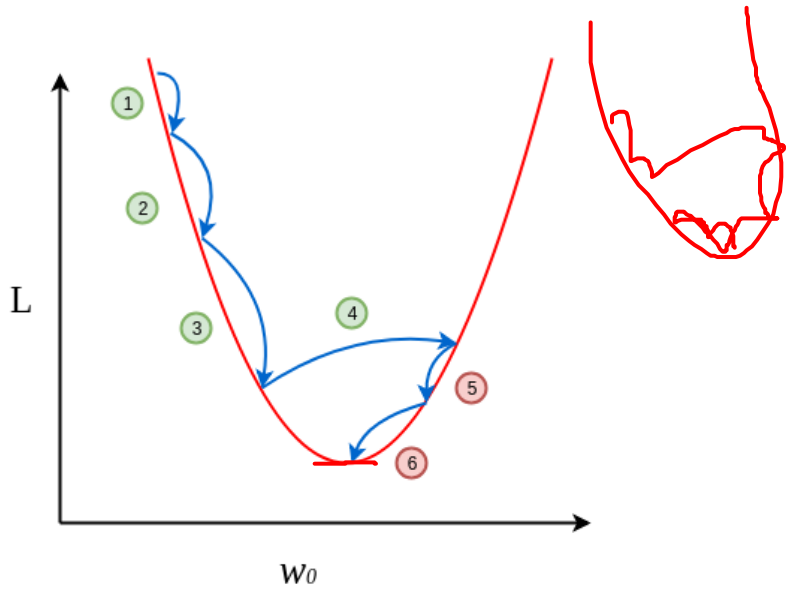
http://proceedings.mlr.press/v28/sutskever13.pdf

NAG

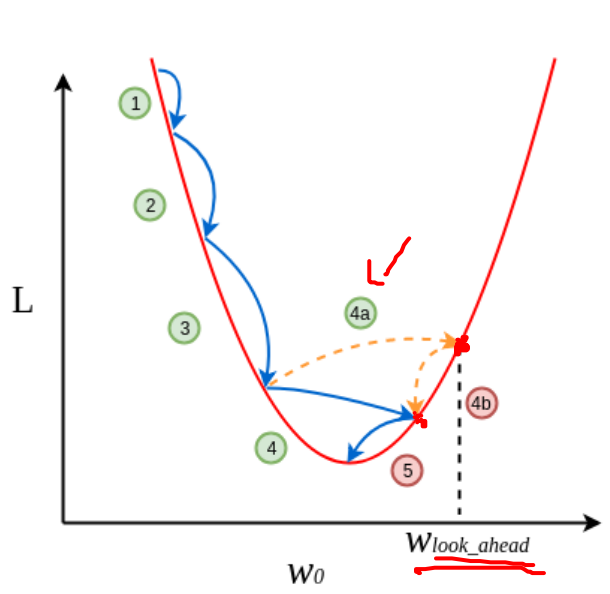
étape de gradient «anticipé»

$$w^* = w_t - \alpha m_{t-1}$$

$$m_t = \beta m_{t-1} + (1-\beta)\frac{\partial L}{\partial w_t}$$



(a) Momentum-Based Gradient Descent



(b) Nesterov Accelerated Gradient Descent

$$\bigcirc \Longrightarrow \frac{\partial L}{\partial w_0} = \frac{Negative(-)}{Positive(+)}$$

$$\bigcirc \Longrightarrow \frac{\partial L}{\partial w_0} = \frac{Negative(-)}{Negative(-)}$$

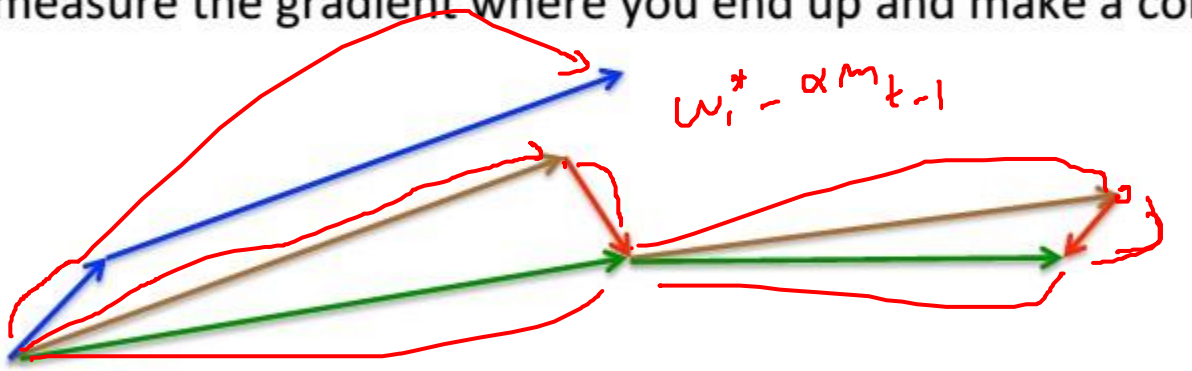
w	Parameters of Network
t	Iteration number
α	Step size/Learning rate
β	Momentum

$$w_{t+1} = w_t - \alpha m_t$$

NAG vs Standard Momentum

A picture of the Nesterov method

- First make a big jump in the direction of the previous accumulated gradient.
- Then measure the gradient where you end up and make a correction.



brown vector = jump, red vector = correction, green vector = accumulated gradient

blue vectors = standard momentum

Exemple code

```python
def nesterov(model, X_train, y_train, minibatch_size):
    velocity = {k: np.zeros_like(v) for k, v in model.items()}
    gamma = .9

    minibatches = get_minibatch(X_train, y_train, minibatch_size)

    for iter in range(1, n_iter + 1):
        idx = np.random.randint(0, len(minibatches))
        X_mini, y_mini = minibatches[idx]

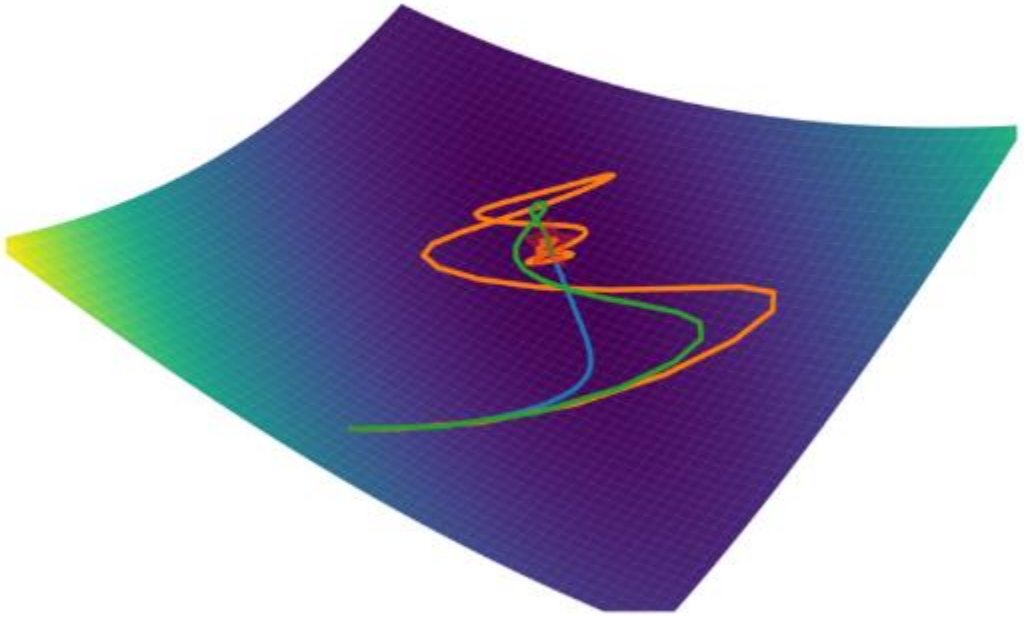
        model_ahead = {k: v + gamma * velocity[k] for k, v in model.items()}
        grad = get_minibatch_grad(model_ahead, X_mini, y_mini)

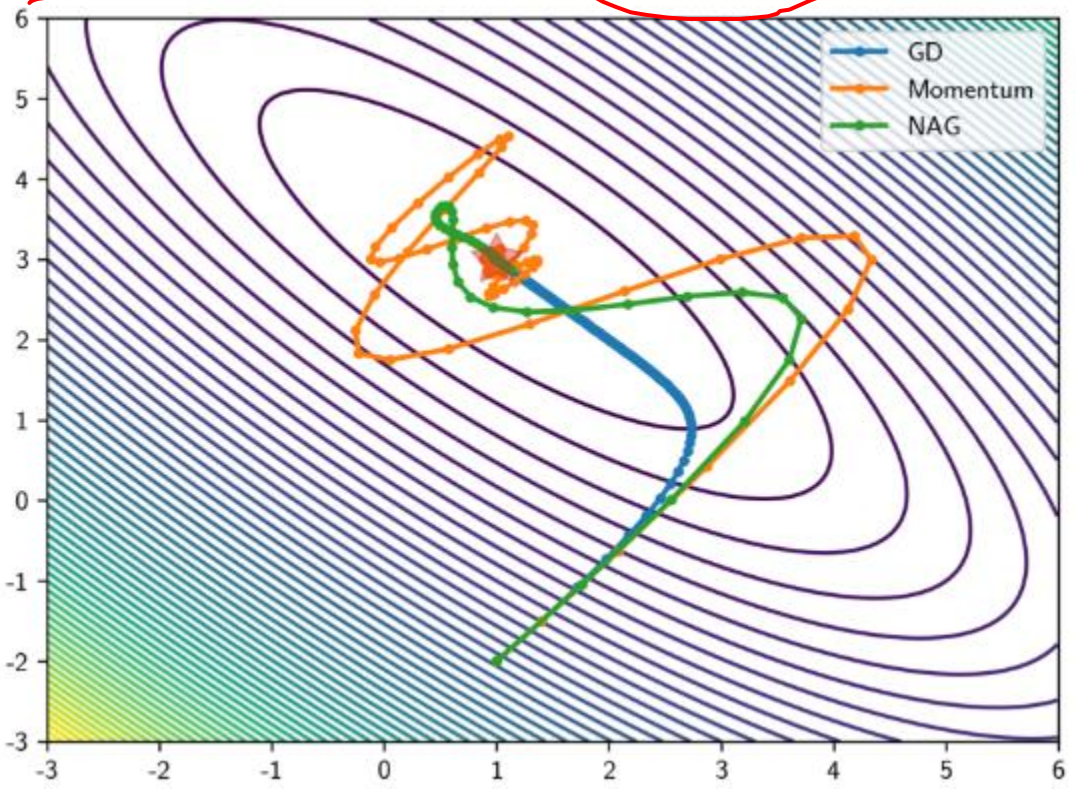
        for layer in grad:
            velocity[layer] = gamma * velocity[layer] + alpha * grad[layer]
            model[layer] -= velocity[layer]

    return model
```

Pourquoi quelque chose de nouveau (au-delà de Momentum / NAG)?

- Comment définir le taux d'apprentissage et la dégradation des taux d'apprentissage?
- Idéalement, des taux d'apprentissage adaptatifs





Pourquoi quelque chose de nouveau (au-delà de Momentum / NAG)?

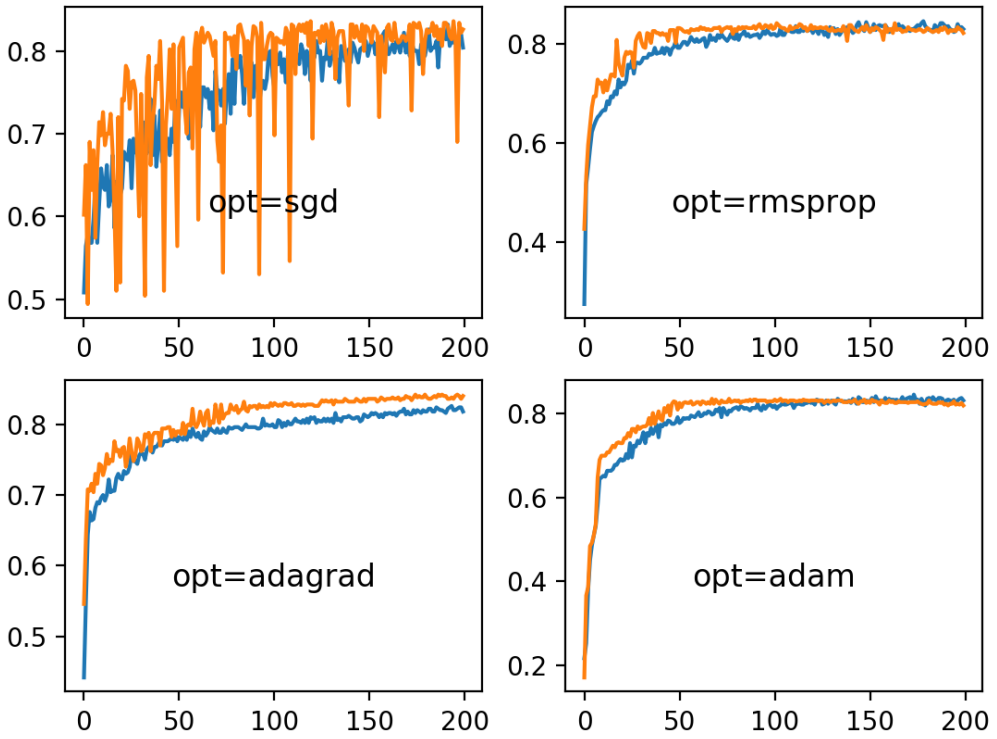
- Les neurones de chaque couche apprennent différemment
  - L'amplitude des gradients varie d'une couche à l'autre
  - Les premières couches obtiennent des “vanishing gradients”
- Devrait idéalement utiliser des taux d'apprentissage adaptatifs séparés
  - One of the reasons for having “gain” or lr multipliers in caffe
- Algorithmes de taux d'apprentissage adaptatif
  - Jacobs 1989 - accord en signe entre gradient actuel pour un poids et la vitesse pour ce poids
  - Utilisez des mini-batch plus grands







Adaptive Learning Rates



Resilient Propagation (Rprop)

- Rprop est un algorithme de descente de gradient populaire qui n'utilise que les signes de dégradés pour calculer les mises à jour.
- Il signifie Propagation résiliente et fonctionne bien dans de nombreuses situations car il adapte la taille de pas de manière dynamique pour chaque poids indépendamment.

1.Rojas, R., 2013. Réseaux de neurones: une introduction systématique. Springer Science & Business Media.

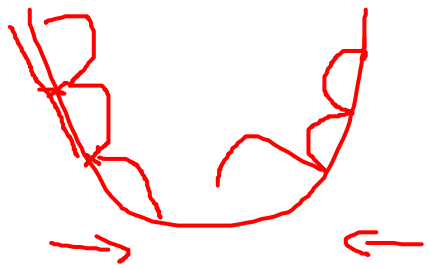
2.Riedmiller, M. et Braun, H., 1993. Une méthode adaptative directe pour un apprentissage de rétropropagation plus rapide: l'algorithme RPROP. Dans Neural Networks, 1993., IEEE International Conference on (pp. 586-591). IEEE.

https://paginas.fe.up.pt/~ee02162/dissertacao/RPROP%20paper.pdf

Resilient Propagation (Rprop)

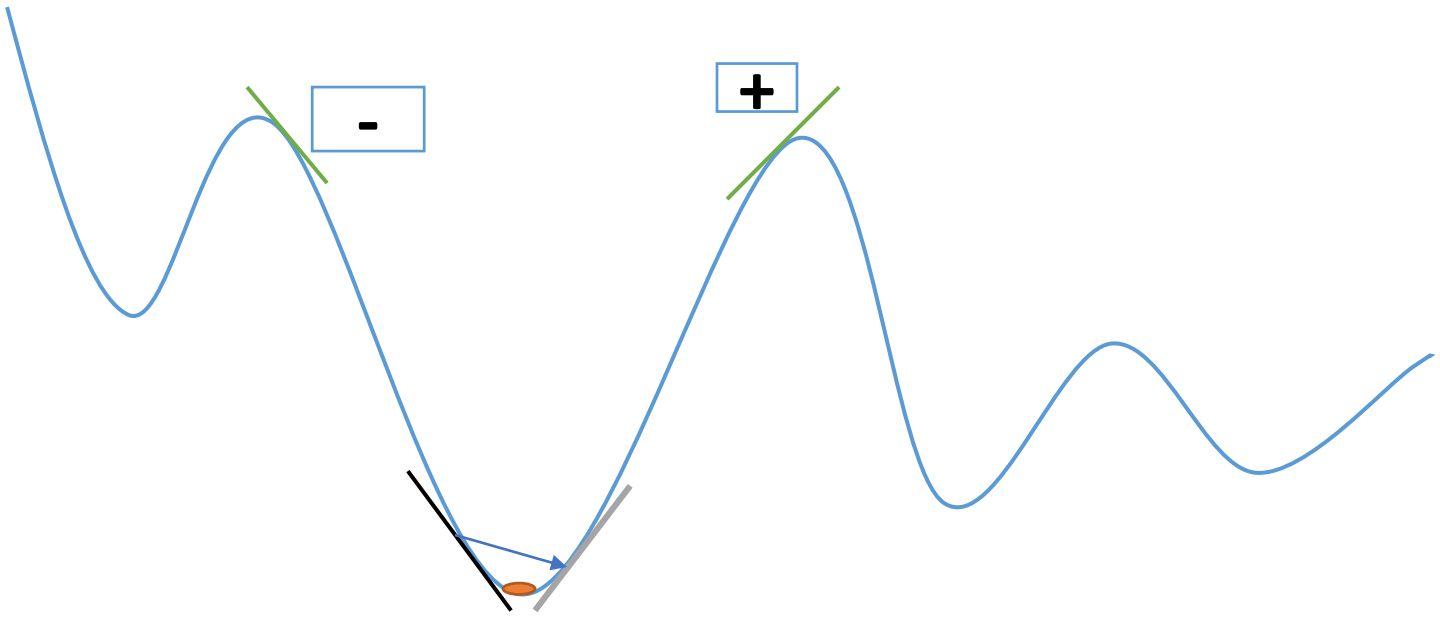
- Riedmiller and Braun 1993
- Aborder le problème du taux d'apprentissage adaptatif

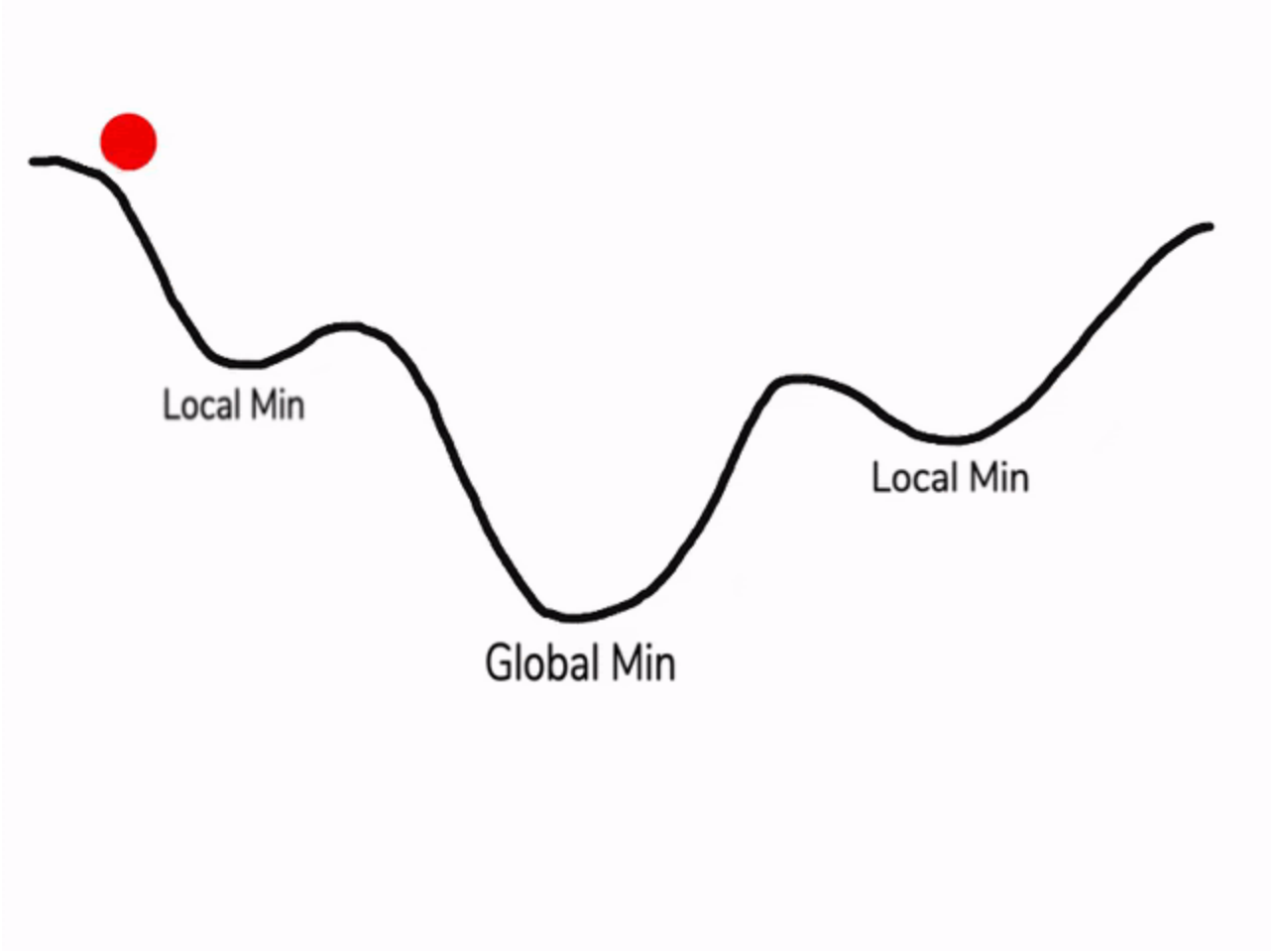
- Augmenter le taux d'apprentissage d'un poids de manière **multiplicative** si les signes des deux derniers gradients sont les mêmes
- Sinon, diminuer le taux d'apprentissage de manière **multiplicative**



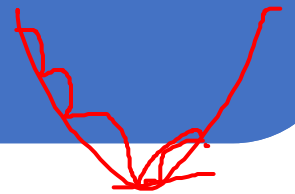


Signes; jumping over the optimum

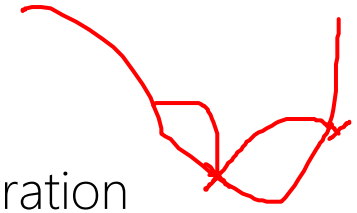



Local Min
Local Min
Global Min

Idée



- À chaque itération de Rprop, les gradients sont calculés et les tailles de pas sont mises à jour individuellement pour chaque dimension. Cela se fait en comparant le signe du gradient de l'itération actuelle et précédente.
- L'idée ici est la suivante:
  - Lorsque les signes sont les mêmes, on va dans le même sens que dans l'itération précédente. Comme cela semble être une bonne direction, la taille du pas doit être augmentée pour atteindre l'optimum plus rapidement
  - Si le signe a changé, la nouvelle mise à jour se déplace dans une direction différente. Cela signifie que nous venons de sauter par-dessus un optimum. La taille du pas doit être diminuée pour éviter de sauter à nouveau par-dessus l'optimum



Rprop Update

Hyperparamètres

- Valeurs populaires pour ***n-*** et ***n+*** sont 1.2 et 0,5.

- Heuristiquement, cela fonctionne bien pour augmenter lentement la taille du pas, tout en permettant la possibilité de la réduire rapidement en sautant autour d'un optimum. Pour affiner les poids, il est important que ***n-*** n'est pas la réciproque de ***n+***, pour permettre de nombreuses tailles de pas différentes.

$$
\begin{aligned}
&\text{if} \quad f'_t f'_{t-1} > 0 \\
&\quad v_t = \eta^+ v_{t-1} \\
&\text{else if} \quad f'_t f'_{t-1} < 0 \\
&\quad v_t = \eta^- v_{t-1} \\
&\text{else} \\
&\quad v_t = v_t
\end{aligned}
$$

$$\theta_{t+1} = \theta_t - v_t$$

$$0 < \eta- < 1 < \eta+$$

tion of the weight update. The *size* of the weight change is exclusively determined by a weight-specific, so-called 'update-value' $\Delta_{ij}^{(t)}$:

$$\Delta w_{ij}^{(t)} = \begin{cases} -\Delta_{ij}^{(t)} & , \text{ if } \frac{\partial E}{\partial w_{ij}}^{(t)} > 0 \\ +\Delta_{ij}^{(t)} & , \text{ if } \frac{\partial E}{\partial w_{ij}}^{(t)} < 0 \\ 0 & , \text{ else} \end{cases} \quad (1)$$

where $\frac{\partial E}{\partial w_{ij}}^{(t)}$ denotes the summed gradient information over all patterns of the pattern set ('batch learning').

It should be noted, that by replacing the $\Delta_{ij}(t)$ by a constant update-value Δ, equation (1) yields the so-called 'Manhattan'-update rule.

The second step of Rprop learning is to determine the new update-values $\Delta_{ij}(t)$. This is based on a sign-dependent adaptation process, similar to the learning-rate adaptation in [4], [5].

$$\Delta_{ij}^{(t)} = \begin{cases} \eta^+ * \Delta_{ij}^{(t-1)} & , \text{ if } \frac{\partial E}{\partial w_{ij}}^{(t-1)} * \frac{\partial E}{\partial w_{ij}}^{(t)} > 0 \\ \eta^- * \Delta_{ij}^{(t-1)} & , \text{ if } \frac{\partial E}{\partial w_{ij}}^{(t-1)} * \frac{\partial E}{\partial w_{ij}}^{(t)} < 0 \\ \Delta_{ij}^{(t-1)} & , \text{ else} \end{cases} \quad (2)$$

where $0 < \eta^- < 1 < \eta^+$

Rprop Update

$$c = \frac{\partial E}{\partial w_{ij}}^{(t)} \cdot \frac{\partial E}{\partial w_{ij}}^{(t-1)}$$

Calculer le changement de poids

$$\Delta w_{ij}^{(t)} = \begin{cases} -\Delta_{ij}^{(t)} & , \text{if } c > 0 \\ +\Delta_{ij}^{(t)} & , \text{if } c < 0 \\ 0 & , \text{otherwise} \end{cases}$$

Modifier les valeurs de mise à jour

$$\Delta_{ij}^{(t)} = \begin{cases} \eta^{+} \cdot \Delta_{ij}^{(t-1)} & , \text{if } c > 0 \\ \eta^{-} \cdot \Delta_{ij}^{(t-1)} & , \text{if } c < 0 \\ \Delta_{ij}^{(t-1)} & , \text{otherwise} \end{cases}$$

Rprop Initialization

- Initialiser toutes les mises à jour à l'itération 0 à une valeur constante
  - If you set both learning rates to 1, you get “Manhattan update rule”

$$v_0 = \delta$$

- Rprop divise efficacement le gradient par son amplitude
  - Vous ne mettez jamais à jour en utilisant le dégradé lui-même, mais par son signe

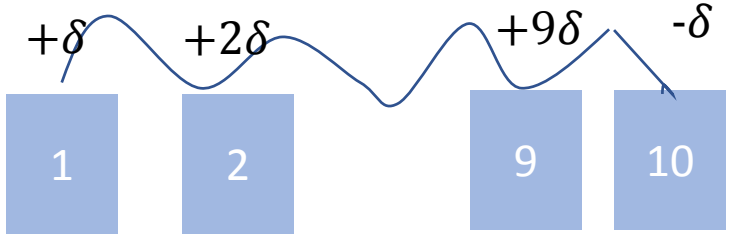
Problems with Rprop

- Considérez un poids qui obtient des mises à jour de 0,1 en neuf mini-lots, et -0,9 dans le dixième mini batch
- SGD garderait ce poids à peu près là où il a commencé,

Rprop augmenterait le poids de neuf fois par δ, puis pour la dixième mise à jour diminuer le poids de δ

  - Mise à jour effective $9\delta - \delta = 8\delta$

- Donc sur plusieurs mini-batches, nous adaptons les mises à jour très différemment



Nous devons faire évoluer les mises à jour sur plusieurs mini-batches de la même manière

➢ Comment ????

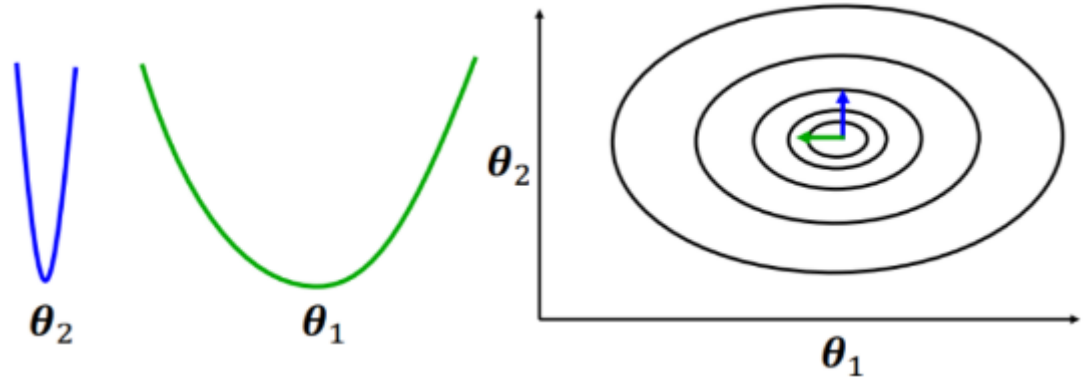
Le gradient adaptatif, AdaGrad

- Le gradient adaptatif, ou AdaGrad (Duchi et al., 2011), agit sur la composante du taux d'apprentissage en divisant le taux d'apprentissage par la racine carrée de **v** , qui est la somme cumulée des gradients carrés actuels et passés (c'est-à-dire jusqu'au temps t) .

- Notez que le composant du gradient reste inchangé comme dans SGD.

Adaptive Gradient (AdaGrad)

- Duchi et al., 2010
- Nous devons faire évoluer les mises à jour sur plusieurs mini-batches de la même manière
- Utiliser les mises à jour du gradient comme indicateur de la mise à l'échelle



$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{v_t + \epsilon}} \cdot \frac{\partial L}{\partial w_t}$$

$$v_t = v_{t-1} + \left[\frac{\partial L}{\partial w_t}\right]^2$$

- Adapter la MAJ de chaque paramètre w_t individuellement pour uniformiser le taux de changement

Exemple code

```python
def adagrad(model, X_train, y_train, minibatch_size):
    cache = {k: np.zeros_like(v) for k, v in model.items()}

    minibatches = get_minibatch(X_train, y_train, minibatch_size)

    for iter in range(1, n_iter + 1):
        idx = np.random.randint(0, len(minibatches))
        X_mini, y_mini = minibatches[idx]

        grad = get_minibatch_grad(model, X_mini, y_mini)

        for k in grad:
            cache[k] += grad[k]**2
            model[k] -= alpha * grad[k] / (np.sqrt(cache[k]) + eps)

    return model
```

$$v_t = v_{t-1} + \left[\frac{\partial L}{\partial w_t}\right]^2$$

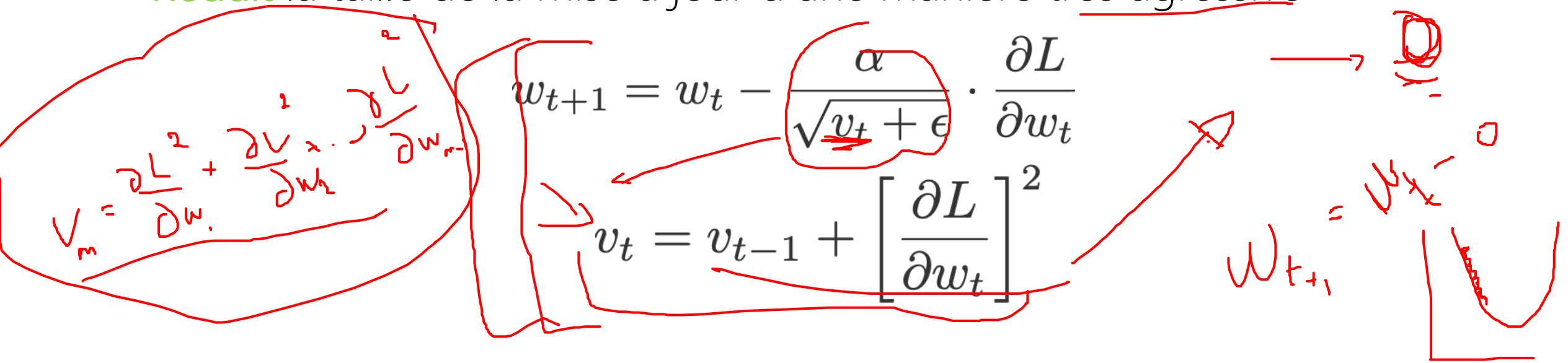
$$w_{t+1} = w_t = \frac{\alpha}{\sqrt{v_t + \epsilon}} \cdot \frac{\partial L}{\partial w_t}$$

Problems with AdaGrad

- Réduit la taille de la mise à jour d'une manière très agressive

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{v_t + \epsilon}} \cdot \frac{\partial L}{\partial w_t}$$

$$v_t = v_{t-1} + \left[\frac{\partial L}{\partial w_t}\right]^2$$



- Le problème avec l'algorithme AdaGrad était que le taux d'apprentissage devient très petit avec un grand nombre d'itérations, ce qui conduit à une *convergence lente*.

RMSProp = Rprop + SGD

- Tieleman & Hinton et al., 2012
- Au lieu de prendre la somme cumulative des gradients carrés comme dans AdaGrad, nous prenons la moyenne mobile exponentielle De ces gradients.
- Mettre à l'échelle les mises à jour de la même manière sur mini-batches
- Décroissant la moyenne du gradient carré
  - Plutôt que la somme des gradients carrés dans AdaGrad

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{v_t + \epsilon}} \cdot \frac{\partial L}{\partial w_t}$$

$$v_t = \beta v_{t-1} + (1-\beta)\left[\frac{\partial L}{\partial w_t}\right]^2$$

et v initialisé à 0.

Valeurs par défaut (de Keras):

- $\alpha = 0{,}001$
- $\beta = 0{,}9$ (recommandé)
- $\varepsilon = 10^{-6}$

RMSProp

- A réussi à former des modèles récurrents
- L'utilisation de Momentum ne montre généralement pas beaucoup d'amélioration

Exemple de code

```python
def rmsprop(model, X_train, y_train, minibatch_size):
    cache = {k: np.zeros_like(v) for k, v in model.items()}
    beta = .9

    minibatches = get_minibatch(X_train, y_train, minibatch_size)

    for iter in range(1, n_iter + 1):
        idx = np.random.randint(0, len(minibatches))
        X_mini, y_mini = minibatches[idx]

        grad = get_minibatch_grad(model, X_mini, y_mini)

        for k in grad:
            cache[k] = beta * cache[k] + (1 - beta) * (grad[k]**2)
            model[k] -= alpha * grad[k] / (np.sqrt(cache[k]) + eps)

    return model
```

$$v_t = \beta v_{t-1} + (1-\beta)\left[\frac{\partial L}{\partial w_t}\right]^2$$

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{v_t + \epsilon}} \cdot \frac{\partial L}{\partial w_t}$$

Fancy RMSProp

- “No more pesky learning rates” – Schaul et al.
  - Computes a diagonal Hessian and uses something similar to RMSProp
  - Diagonal Hessian computation requires an additional Forward-Backward pass
    - Double the time of SGD

Adadelta

Le problème avec l'algorithme AdaGrad était que le taux d'apprentissage devient très petit avec un grand nombre d'itérations, ce qui conduit à une *convergence lente*.
Pour éviter cela, l'algorithme AdaDelta a l'idée de prendre *une moyenne en décroissance exponentielle*.

Adadelta

- Adadelta est une extension d'Adagrad et tente également de réduire l'agressivité d'Adagrad, en réduisant de manière monotone le taux d'apprentissage
- Pour ce faire, il restreint la fenêtre du gradient accumulé passé à une taille fixe de w. La moyenne mobile au temps t dépend alors de la moyenne précédente et du gradient actuel
- En Adadelta, nous n'avons pas besoin de définir le taux d'apprentissage par défaut car nous prenons le rapport de la moyenne mobile des pas de temps précédents au gradient actuel.

Adadelta

- Comme RMSprop, Adadelta (Zeiler, 2012) est également une autre amélioration d'AdaGrad, en se concentrant sur la composante du *taux d'apprentissage*.

- La différence entre Adadelta et RMSprop est qu'Adadelta supprime complètement l'utilisation du paramètre de taux d'apprentissage en le remplaçant par *D*, la moyenne mobile exponentielle des deltas carrés .

AdaDelta

- Zeiler et al., 2012
- Obtenez des mises à jour qui correspondent aux unités
- Conserver les propriétés de RMSProp

$$w_{t+1} = w_t - \frac{\sqrt{D_{t-1} + \epsilon}}{\sqrt{v_t + \epsilon}} \cdot \frac{\partial L}{\partial w_t}$$

Tel que:

$$D_t = \beta D_{t-1} + (1 - \beta)[\Delta w_t]^2$$

$$v_t = \beta v_{t-1} + (1 - \beta)\left[\frac{\partial L}{\partial w_t}\right]^2$$

avec D et v initialisés à 0, et

$$\Delta w_t = w_t - w_{t-1}$$

- Valeurs par défaut (de Keras):
- β = 0,95
- ε = 10^{-6}

Adam

- L'estimation adaptative du moment, ou Adam (Kingma & Ba, 2014), est simplement une combinaison de momentum et de RMSprop. Il agit sur
  - la composante de gradient en utilisant m , la moyenne mobile exponentielle des gradients (comme dans momentum), et
  - la composante du taux d'apprentissage en divisant le taux d'apprentissage α par la racine carrée de v , la moyenne mobile exponentielle des gradients carrés (comme dans RMSprop).

Adam

- Kingma & Ba, 2015
- Averages of gradient, or squared gradients
- Bias correction

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\frac{\partial L}{\partial w_t}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\left[\frac{\partial L}{\partial w_t}\right]^2$$

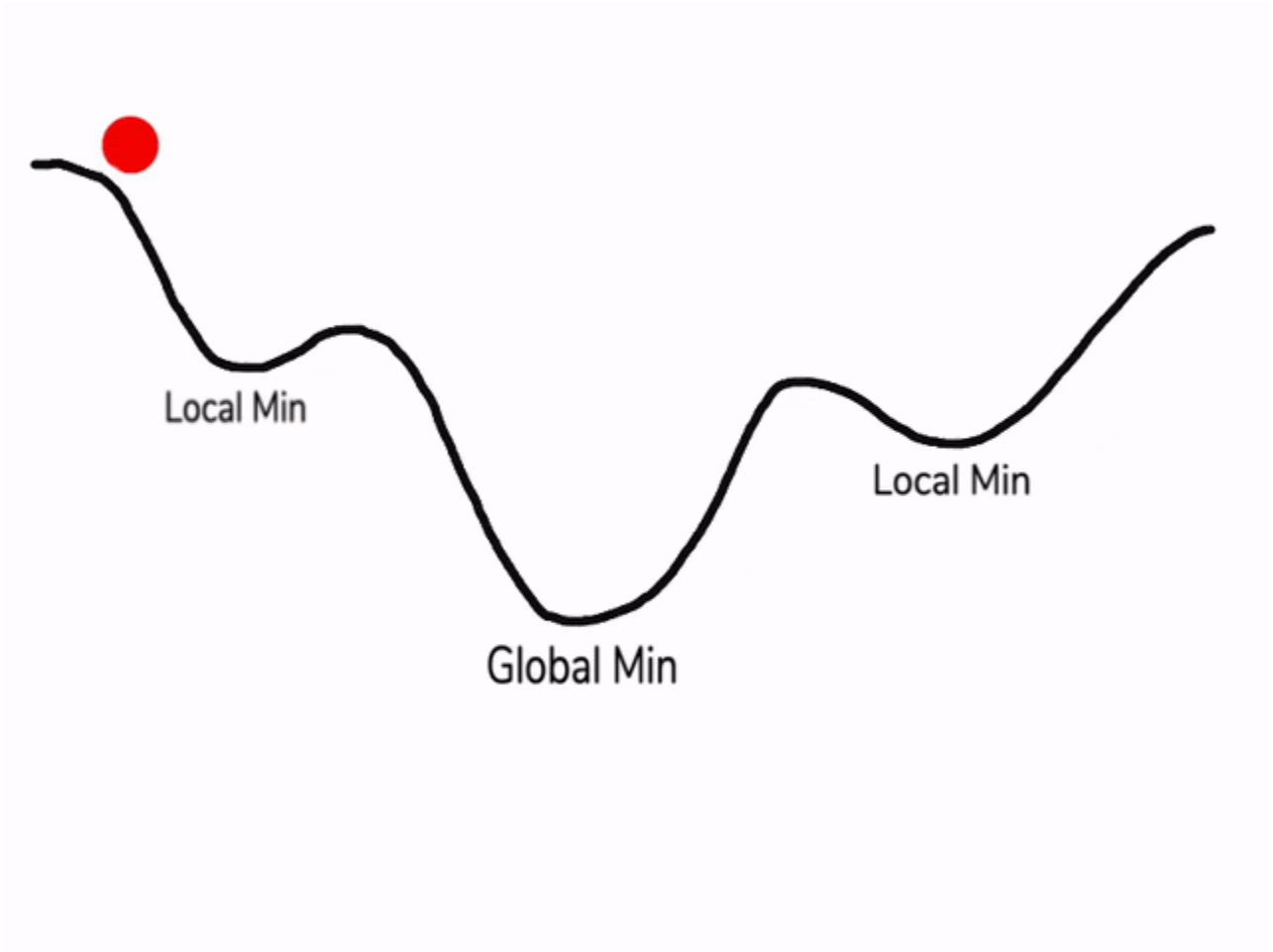
$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \cdot \hat{m}_t$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

bias corrections

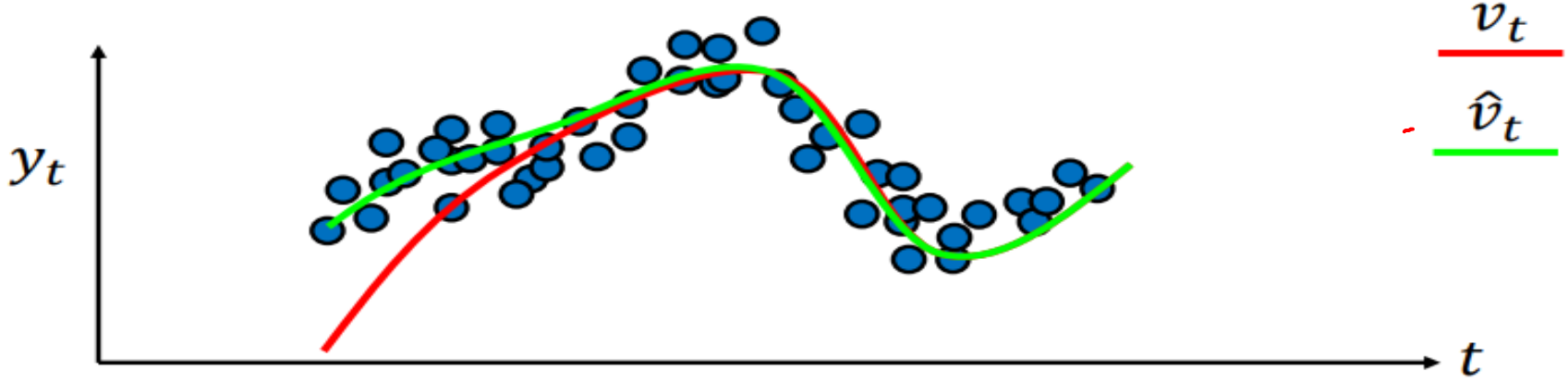
avec *m* et *v* initialisés à 0.


Local Min
Local Min
Global Min

Adam

- Solution pour résoudre les inconvénients de RMSprop + momentum / Nesterov.
- Méthode:
  - Calculer une estimation du premier et second moments avec une moyenne mobile exponentielle à taux $\beta_1, \beta_2 \in 0, 1$.
    - Valeurs suggérées: $\beta_1 = 0.9$ et $\beta_2 = 0.999$
  - Corriger les biais des moments.
  - Le premier moment normalisé par le second moment donne la direction de MAJ.

Adam



$$v_t = \rho v_{t-1} + (1-\rho) y_t$$

$$v_0 = 0$$
$$v_1 = 0.99 \cdot v_0 + 0.01 \cdot y_1$$
$$= 0.01 \cdot y_1$$
$$v_2 = 0.99 \cdot v_1 + 0.01 \cdot y_2$$
$$= 0.0099 \cdot y_1 + 0.01 \cdot y_2$$

$$\hat{v}_t = \frac{v_t}{1 - (\rho)^t}$$

$$\hat{v}_2 = \frac{v_2}{1 - 0.99^2}$$
$$= \frac{0.0099 \cdot y_1 + 0.01 \cdot y_2}{0.0199}$$
$$= 0.497 \cdot y_1 + 0.503 \cdot y_2$$

Adam update rule

l'optimiseur Adam peut aussi s'écrire:

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon}\left(\beta_1 \hat{m}_{t-1} + \frac{1-\beta_1}{1-\beta_1^t} \cdot \frac{\partial L}{\partial w_t}\right)$$

Mise à jour du poids pour l'optimiseur Adam

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\frac{\partial L}{\partial w_t}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\left[\frac{\partial L}{\partial w_t}\right]^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \cdot \hat{m}_t$$

Mise à jour du poids pour l'optimiseur Adam

Adam

- Algorithme
  1. Échantillonage: $(\boldsymbol{x}^{(i)}, y^{(i)}) \sim \hat{p}_{data}, 1 \leq i \leq m.$
  2. Gradient: $\hat{\boldsymbol{g}} = \frac{1}{m} \sum_{i=1}^{m} \nabla_{\boldsymbol{\theta}} L\left(f\left(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}\right), y^{(i)}\right)$
  3. 1er moment: $\boldsymbol{s} \leftarrow \rho_1 \boldsymbol{s} + (1 - \rho_1)\hat{\boldsymbol{g}}$
  4. 2e moment: $\boldsymbol{r} \leftarrow \rho_2 \boldsymbol{r} + (1 - \rho_2)\hat{\boldsymbol{g}} \odot \hat{\boldsymbol{g}}$
  5. Correction biais 1er moment : $\hat{\boldsymbol{s}} \leftarrow \frac{\boldsymbol{s}}{1 - (\rho_1)^t}$
  6. Correction biais 2e moment : $\hat{\boldsymbol{r}} \leftarrow \frac{\boldsymbol{r}}{1 - (\rho_2)^t}$
  7. Direction: $\boldsymbol{d} \leftarrow \epsilon_t \frac{\hat{\boldsymbol{s}}}{\sqrt{\hat{\boldsymbol{r}}} + \delta}$
  8. MAJ: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \boldsymbol{d}$

Exemple de code

```python
def adam(model, X_train, y_train, minibatch_size):
    M = {k: np.zeros_like(v) for k, v in model.items()}
    R = {k: np.zeros_like(v) for k, v in model.items()}
    beta1 = .9
    beta2 = .999

    minibatches = get_minibatch(X_train, y_train, minibatch_size)

    for iter in range(1, n_iter + 1):
        t = iter
        idx = np.random.randint(0, len(minibatches))
        X_mini, y_mini = minibatches[idx]

        grad = get_minibatch_grad(model, X_mini, y_mini)

        for k in grad:
            M[k] = beta1 * M[k] + (1. - beta1) * grad[k]
            R[k] = beta2 * R[k] + (1. - beta2) * grad[k]**2

            m_k_hat = M[k] / (1. - beta1**(t))
            r_k_hat = R[k] / (1. - beta2**(t))

            model[k] -= alpha * m_k_hat / (np.sqrt(r_k_hat) + eps)

    return model
```

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\frac{\partial L}{\partial w_t}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\left[\frac{\partial L}{\partial w_t}\right]^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{v}_t}+\epsilon}\cdot \hat{m}_t$$

AdaMax

- AdaMax (Kingma & Ba, 2015) est une adaptation de l'optimiseur Adam par les mêmes auteurs en utilisant des normes à l'infini (d'où «max»).
- m est la moyenne mobile exponentielle des gradients, et v est la moyenne mobile exponentielle de la p- normale passée des gradients, approximée à la fonction max

AdaMax

$$w_{t+1} = w_t - \frac{\alpha}{v_t} \cdot \hat{m}_t$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\frac{\partial L}{\partial w_t}$$

$$v_t = \max(\beta_2 v_{t-1}, \left|\frac{\partial L}{\partial w_t}\right|)$$

Nadam

- Nadam (Dozat, 2015) est un acronyme pour Nesterov et Adam optimiser.
- Nadam utilise Nesterov pour mettre à jour le gradient avec une longueur d'avance en remplaçant le m^ précédent dans l'équation ci-dessus par le m^ actuel :

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon}\left(\beta_1 \hat{m}_t + \frac{1-\beta_1}{1-\beta_1^t} \cdot \frac{\partial L}{\partial w_t}\right)$$

AMSGrad

- Une autre variante d'Adam est l'AMSGrad (Reddi et al., 2018).
- Cette variante revisite le composant de taux d'apprentissage adaptatif dans Adam et le modifie pour s'assurer que le courant v est toujours plus grand que le v du pas de temps précédent .

AMSGrad

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \cdot m_t$$

$$\hat{v}_t = \max(\hat{v}_{t-1}, v_t)$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \frac{\partial L}{\partial w_t}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \left[\frac{\partial L}{\partial w_t} \right]^2$$

avec *m* et *v* initialisés à 0.

Vanilla SGD

$$w_{t+1} = w_t - \alpha \frac{\partial L}{\partial w_t}$$

Momentum

$$w_{t+1} = w_t - \alpha m_t$$

$$m_t = \beta m_{t-1} + (1-\beta)\frac{\partial L}{\partial w_t}$$

Adagrad

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{v_t + \epsilon}} \cdot \frac{\partial L}{\partial w_t}$$

$$v_t = v_{t-1} + \left[\frac{\partial L}{\partial w_t}\right]^2$$

RMSprop

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{v_t + \epsilon}} \cdot \frac{\partial L}{\partial w_t}$$

$$v_t = \beta v_{t-1} + (1-\beta)\left[\frac{\partial L}{\partial w_t}\right]^2$$

Adadelta

$$w_{t+1} = w_t - \frac{\sqrt{D_{t-1} + \epsilon}}{\sqrt{v_t + \epsilon}} \cdot \frac{\partial L}{\partial w_t}$$

$$D_t = \beta D_{t-1} + (1-\beta)[\Delta w_t]^2$$

$$v_t = \beta v_{t-1} + (1-\beta)\left[\frac{\partial L}{\partial w_t}\right]^2$$

Nesterov

$$w_{t+1} = w_t - \alpha m_t$$

$$m_t = \beta m_{t-1} + (1-\beta)\frac{\partial L}{\partial w^*}$$

$$w^* = w_t - \alpha m_{t-1}$$

Adam

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{v}_t + \epsilon}} \cdot \hat{m}_t$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\frac{\partial L}{\partial w_t}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\left[\frac{\partial L}{\partial w_t}\right]^2$$

AdaMax

$$w_{t+1} = w_t - \frac{\alpha}{v_t} \cdot \hat{m}_t$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\frac{\partial L}{\partial w_t}$$

$$v_t = \max(\beta_2 v_{t-1}, \left|\frac{\partial L}{\partial w_t}\right|)$$

Nadam

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{v}_t + \epsilon}}\left(\beta_1 \hat{m}_{t-1} + \frac{1-\beta_1}{1-\beta_1^t} \cdot \frac{\partial L}{\partial w_t}\right)$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\frac{\partial L}{\partial w_t}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\left[\frac{\partial L}{\partial w_t}\right]^2$$

AMSGrad

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{v}_t + \epsilon}} \cdot m_t$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\frac{\partial L}{\partial w_t}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\left[\frac{\partial L}{\partial w_t}\right]^2$$

$$\hat{v}_t = \max(\hat{v}_{t-1}, v_t)$$

Adam Results – Logistic Regression

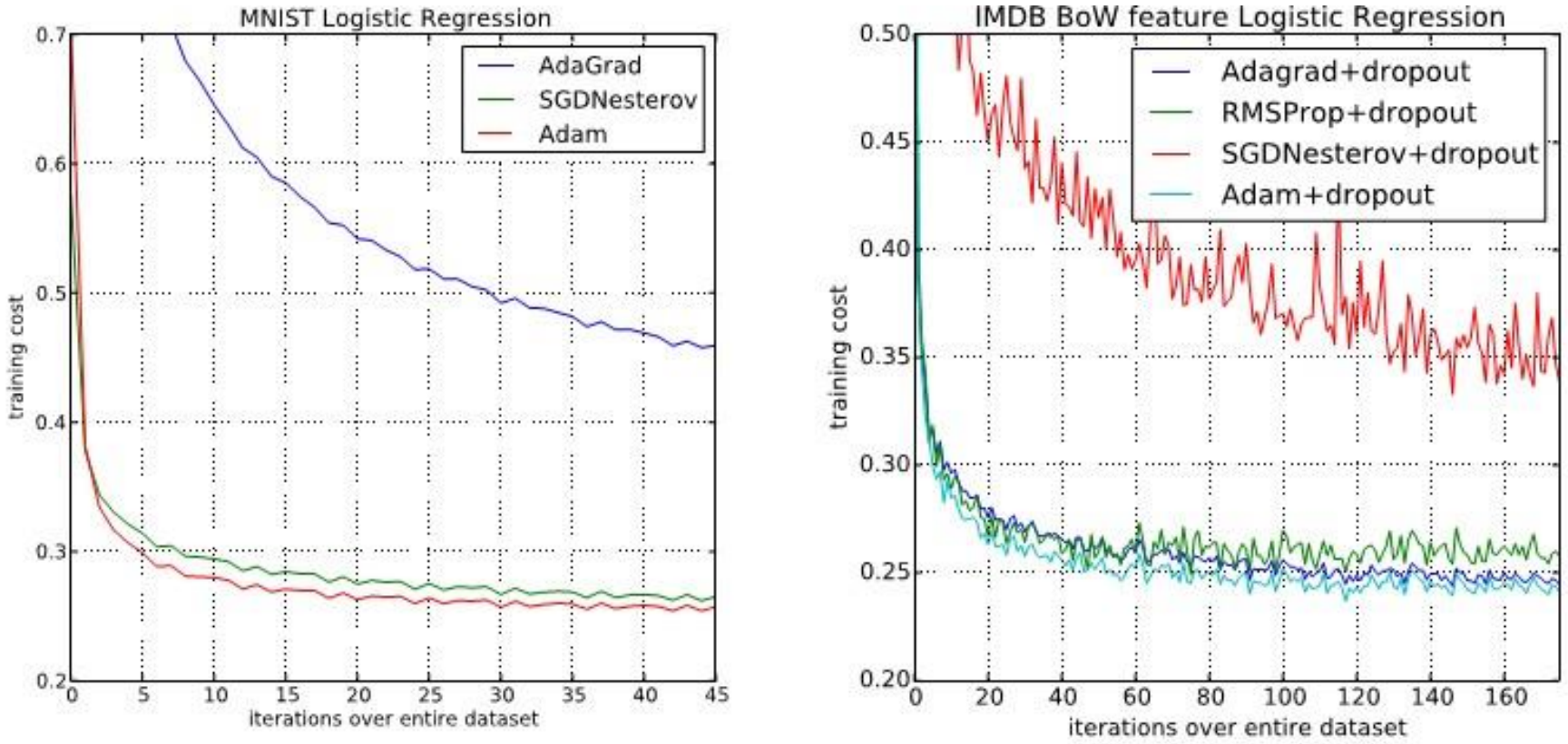
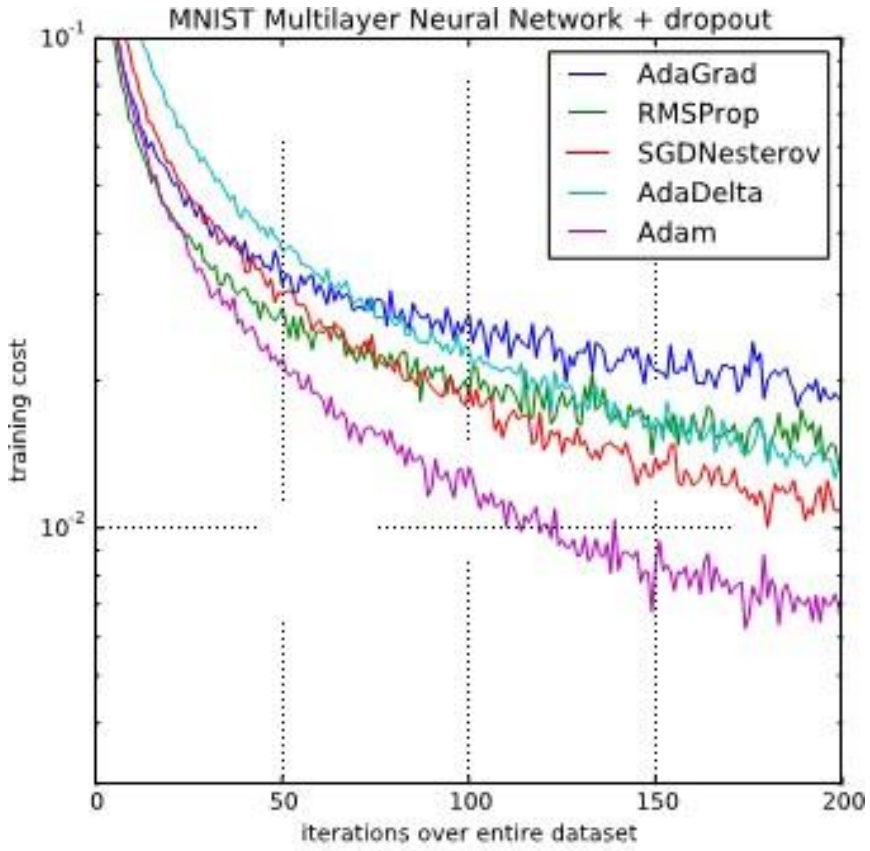


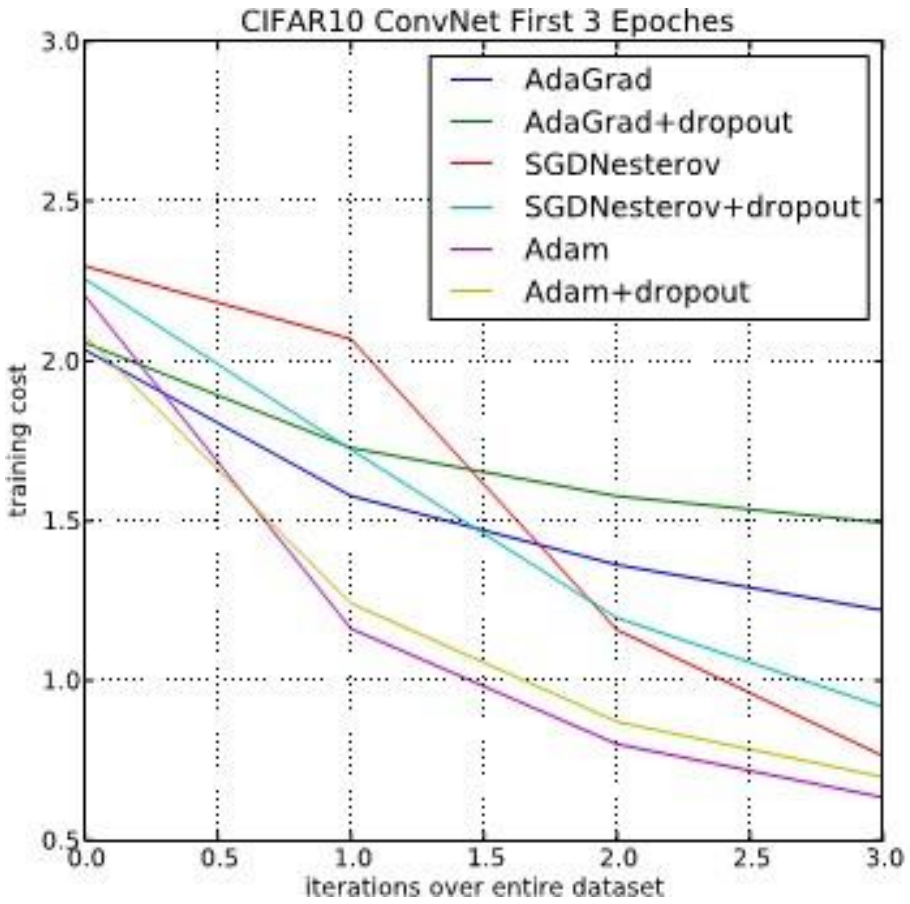
Figure 1: Logistic regression training negative log likelihood on MNIST images and IMDB movie reviews with 10,000 bag-of-words (BoW) feature vectors.

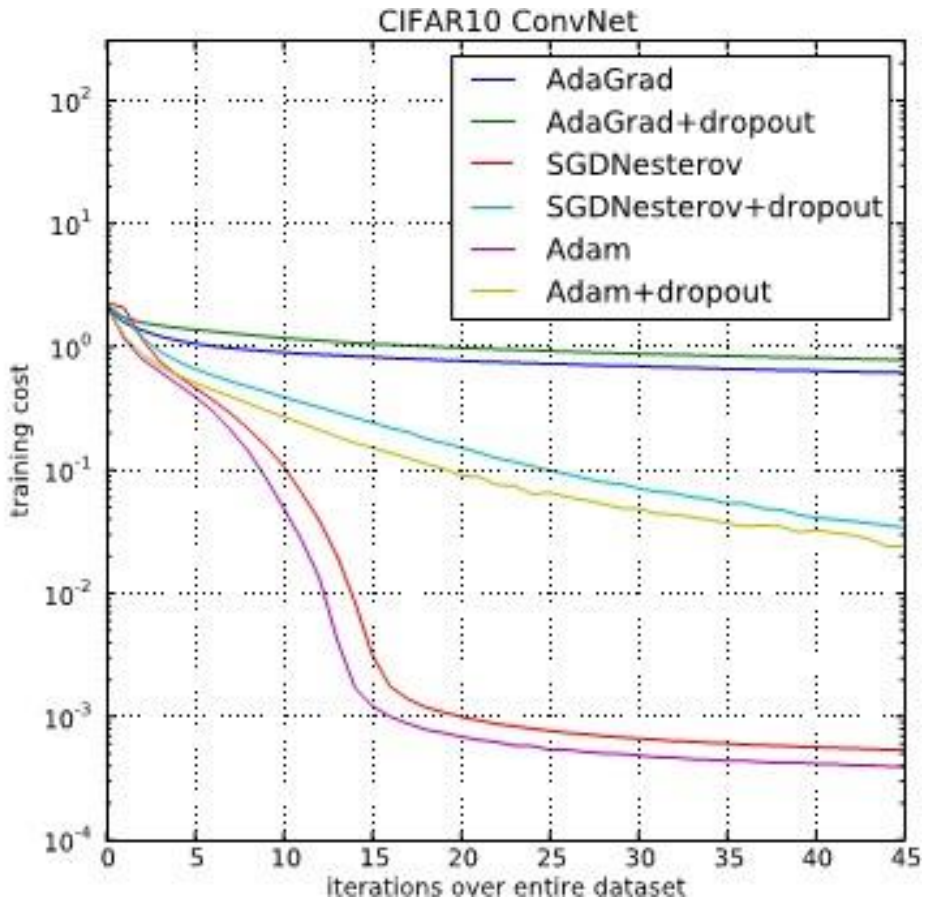
Adam Results - MLP



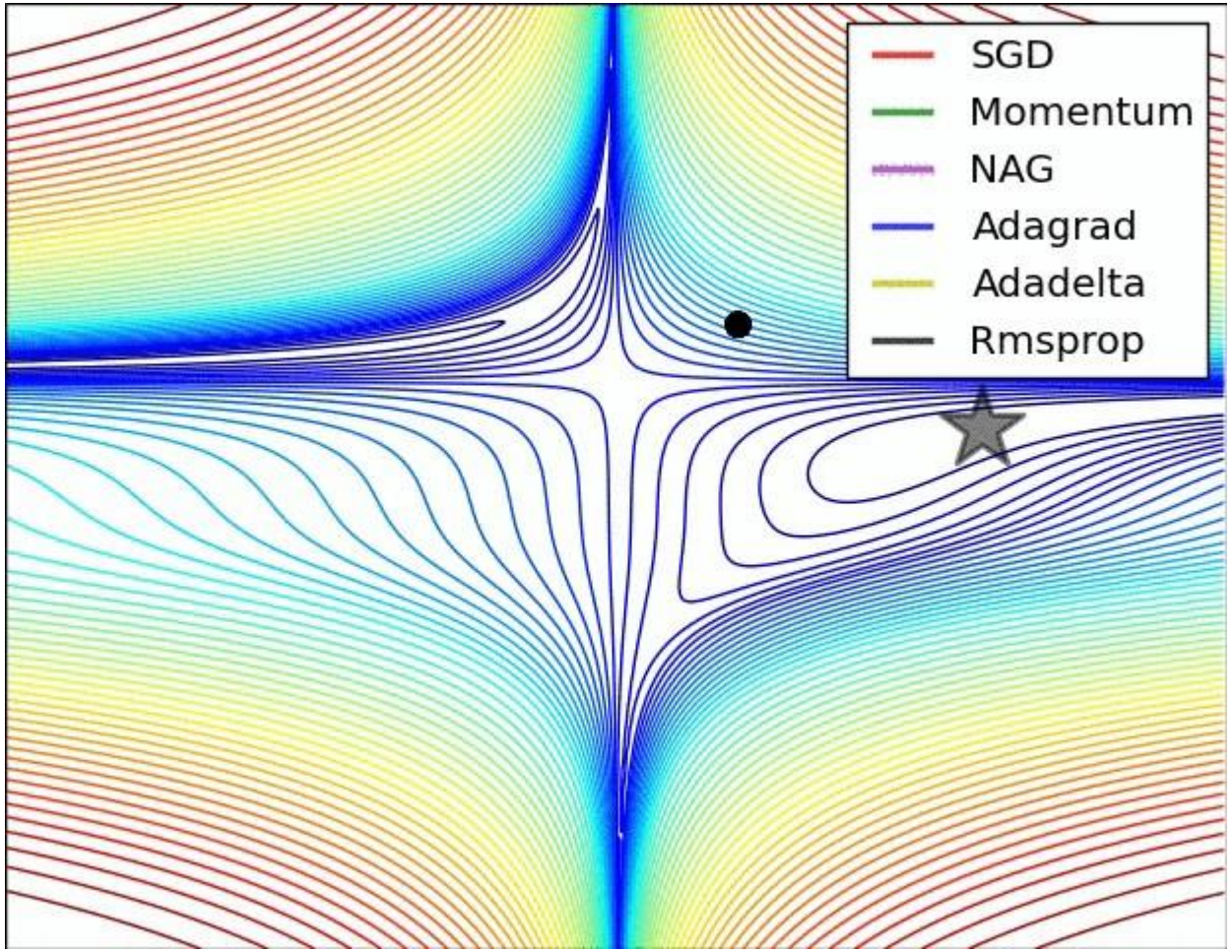
(a)

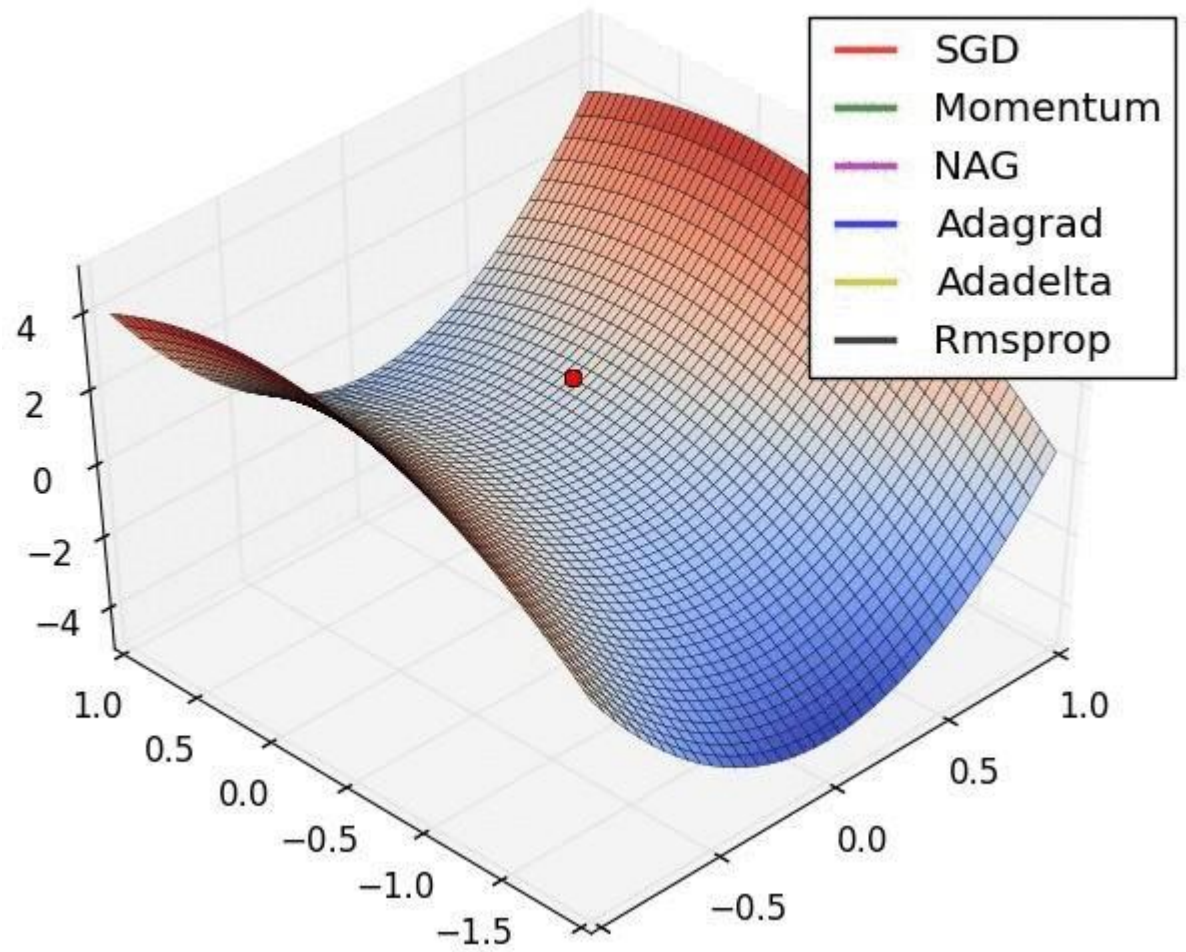
Adam Results – Conv Nets





Visualization





Alec Radford

Comparaison

Approche	Hyper-paramètre	Utilisation mémoire
Batch	ϵ_t	D
SGD	ϵ_t, m	D
SGD + Momentum	ϵ_t, m, α	D*2
SGD + Nesterov	ϵ_t, m, α	D*2
Adagrad	ϵ_t, m, δ	D*2
RMSProp	ϵ_t, m, ρ	D*2
RMSProp + Momentum	$\epsilon_t, m, \rho, \alpha$	D*3
RMSProp + Nesterov	$\epsilon_t, m, \rho, \alpha$	D*3
Adam	$\epsilon_t, m, \rho_1, \rho_2, \delta$	D*3

PyTorch documentation

Batch Normalization

Ioffe and Szegedy, 2015

Batch Normalization

- Le comportement des algorithmes d'apprentissage automatique peut changer lorsque la distribution d'entrée change: si le train et les ensembles de test proviennent de sources entièrement différentes, les distributions seraient différentes.

Batch Normalization

- Dans le contexte du deep learning, nous sommes particulièrement concernés par le changement de la répartition des entrées vers les nœuds internes d'un réseau.
- Un réseau de neurones modifie les poids de chaque couche au cours de la formation. Cela signifie que les activations de chaque couche changent également .
- Puisque les activations d'une couche précédente sont les entrées de la couche suivante, chaque couche du réseau neuronal est confrontée à une situation où la distribution d'entrée change à chaque étape .
- Ceci est problématique car cela oblige chaque couche intermédiaire à s'adapter en permanence à ses entrées changeantes.

Distribution of input

- Having a fixed input distribution is known to help training of linear classifiers
  - Normalize inputs for SVM
  - Normalize inputs for Deep Networks

Distribution of input at each layer

- la distribution des entrées de chaque couche change pendant l'entraînement, à mesure que les paramètres des couches précédentes changent.

- Chaque couche bénéficierait si son entrée avait une distribution constante

Normalize the input to each layer!

- Pour chaque fonctionnalité, la normalisation par lots calcule la moyenne et la variance de cette fonctionnalité dans le mini-lot. Il soustrait ensuite la moyenne et divise l'entité par son écart type du mini-lot. Résultat de la transformation des entrées en moyenne 0 et variance unitaire.

- C'est une technique pour améliorer la vitesse, les performances et la stabilité des réseaux de neurones artificiels. Il est utilisé pour normaliser la couche d'entrée en ajustant et en mettant à l'échelle les activations.

- La normalisation par lots nous permet d'utiliser des taux d'apprentissage beaucoup plus élevés et d'être moins prudents lors de l'initialisation. Il agit également comme un régularisateur, éliminant dans certains cas le besoin d'abandonner.

Normalize the input to each layer!

$$\mu_x = \frac{1}{m}\sum_{i=0}^{m} x_i$$

$$\sigma_x^2 = \frac{1}{m}\sum_{i=0}^{m}(x_i - \mu_x)^2$$

$$\hat{x_i} = \frac{x_i - \mu_x}{\sigma_x}$$

Is this normalization a good idea?

- Considérez les entrées d'une couche sigmoïde
- S'il est normalisé, le sigmoïde peut ne jamais «saturer»

Modify normalization …

- Accommodate identity transform

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma\widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

$$\mu_x = \frac{1}{m}\sum_{i=0}^{m} x_i$$

$$\sigma_x^2 = \frac{1}{m}\sum_{i=0}^{m}(x_i - \mu_x)^2$$

$$\hat{x}_i = \frac{x_i - \mu_x}{\sigma_x}$$

$$y_i = \gamma\hat{x}_i + \beta$$

γ, β Learned parameters

Batch Normalization Results

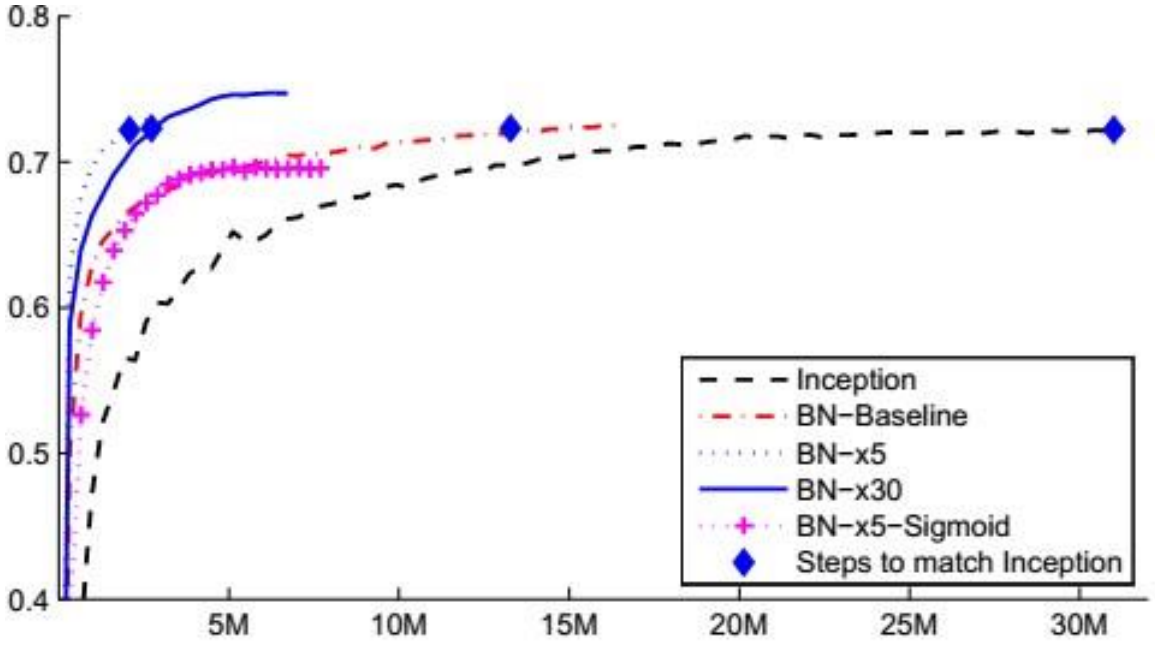


Figure 2: *Single crop validation accuracy of Inception and its batch-normalized variants, vs. the number of training steps.*

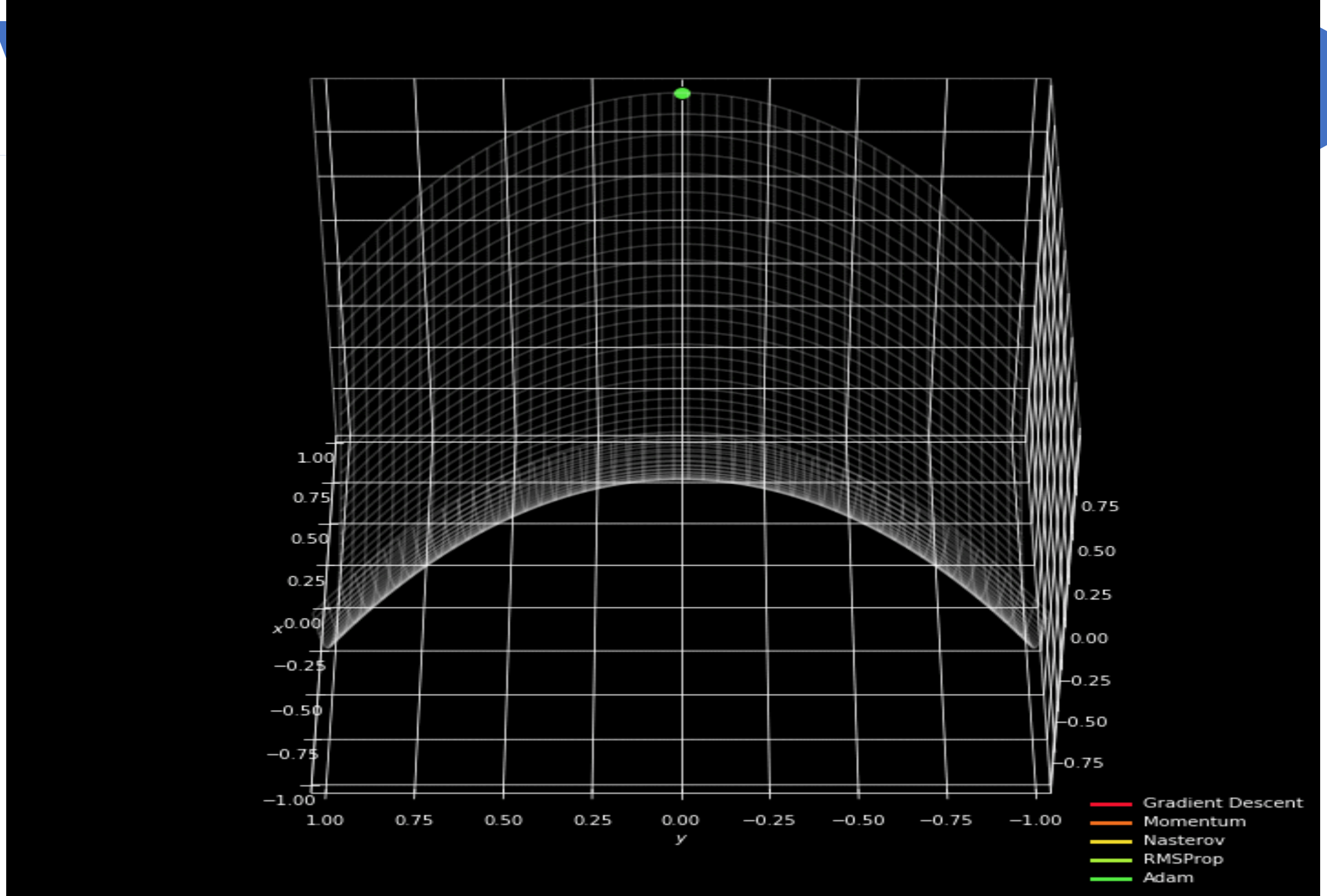
Model	Steps to 72.2%	Max accuracy
Inception	$31.0 \cdot 10^6$	72.2%
BN-Baseline	$13.3 \cdot 10^6$	72.7%
BN-x5	$2.1 \cdot 10^6$	73.0%
BN-x30	$2.7 \cdot 10^6$	74.8%
BN-x5-Sigmoid		69.8%

Figure 3: *For Inception and the batch-normalized variants, the number of training steps required to reach the maximum accuracy of Inception (72.2%), and the maximum accuracy achieved by the network.*

Batch Normalization for ensembles

Model	Resolution	Crops	Models	Top-1 error	Top-5 error
GoogLeNet ensemble	224	144	7	-	6.67%
Deep Image low-res	256	-	1	-	7.96%
Deep Image high-res	512	-	1	24.88	7.42%
Deep Image ensemble	variable	-	-	-	5.98%
BN-Inception single crop	224	1	1	25.2%	7.82%
BN-Inception multicrop	224	144	1	21.99%	5.82%
BN-Inception ensemble	224	144	6	20.1%	**4.9%***

Figure 4: *Batch-Normalized Inception comparison with previous state of the art on the provided validation set comprising 50000 images. *BN-Inception ensemble has reached 4.82% top-5 error on the 100000 images of the test set of the ImageNet as reported by the test server.*


1.00
0.75
0.50
0.25
x 0.00
−0.25
−0.50
−0.75
−1.00
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
y
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
Gradient Descent
Momentum
Nasterov
RMSProp
Adam